UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies

LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUȚINCU

Absolvent:
George-Alexandru MIRON

Iași, 2026

UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies

Distributed Platform for Running Services Within a Cluster

LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUȚINCU

Absolvent:
George-Alexandru MIRON

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII LUCRĂRII DE DISERTAȚIE

Subsemnatul ____,

legitimat cu ____ seria ______ nr. _________ , CNP____________________________

autorul lucrării ____

____

____

elaborată în vederea susținerii examenului de finalizare a studiilor de masterat, programul de studii ______________________________________ organizat de către Facultatea de Automatică şi Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi" din Iaşi, sesiunea ______________ a anului universitar ______________, luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi" din Iaşi (Manualul Procedurilor, UTI.POM.02 - Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) şi a convențiilor internaționale privind drepturile de autor.

Data　　　　　　　　　　　　　　　　　　Semnătura

Cuprins

Distributed Platform for Running Services Within a Cluster

George-Alexandru MIRON

Abstract

This thesis presents the design and implementation of a distributed platform for running containerized services within a cluster. The platform is built from scratch in Go and addresses the core challenges of cluster management: automatic node discovery using UDP multicast heartbeats, resource pool allocation, pluggable load balancing (Least Connections and Weighted strategies), fault tolerance with automatic container recovery and split-brain prevention, JWT-based authentication with refresh tokens, role-based access control and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol for high availability. Worker nodes run Docker containers and communicate with the master through heartbeats and TCP commands. The platform provides a REST API for deploying services, managing nodes and querying the audit log. Compared to production platforms like Kubernetes, the proposed solution focuses on simplicity and clarity, implementing the fundamental distributed systems concepts in an understandable way suitable for educational use and smaller deployments. The platform was validated through unit tests covering the load balancer, scheduler and access control components, as well as manual end-to-end tests demonstrating node discovery, service deployment, fault recovery and security enforcement.

Keywords: distributed systems, cluster management, container orchestration, heartbeat, fault tolerance, load balancing, Raft consensus

Capitolul 1. Introduction

Running services reliably across multiple machines is a common challenge in modern software systems. As user traffic grows, a single server can no longer handle all requests. The application must be split across a cluster of machines, where each machine is called a node.

Managing a cluster brings several problems. The system needs to know which nodes are available. When a new service is deployed, it must decide which node should run it. If a node crashes, the services running on it must be moved to another node. The system also needs access control and a record of who did what and when.

Kubernetes is the most popular platform for solving these problems, but it has hundreds of components and requires a team of engineers to run. This thesis proposes a simpler alternative: a distributed platform built from scratch that implements the fundamental concepts (heartbeats, scheduling, load balancing, fault tolerance, security and audit) in a clear and understandable way.

1.1. Objectives

The main objective of this thesis is to design and implement a distributed platform for running containerized services within a cluster. The platform addresses the following specific objectives:

- **Automatic node discovery**. Worker nodes are discovered automatically by the master when they send their first heartbeat via UDP multicast. No explicit join or leave mechanism is needed.
- **Resource pools**. Nodes are grouped into pools based on their hardware resources (CPU, RAM). Services can be deployed to a specific pool, ensuring they run on appropriate hardware.
- **Load balancing**. When deploying a service, the platform selects the best node using a pluggable load balancing strategy (Least Connections or Weighted).
- **Fault tolerance**. The platform detects node failures through heartbeat monitoring and automatically restarts affected containers on healthy nodes. A Raft consensus protocol ensures master node availability.
- **Security**. User authentication is handled through JWT tokens and a role-based access control system (Admin, Writer, Reader) restricts what each user can do.
- **Audit**. All important actions are logged in an append-only audit log that can be queried through the API.

1.2. Methodology and Tools

The platform is implemented in Go, chosen for its built-in concurrency support (goroutines), efficient networking and suitability for systems programming. The main tools and libraries used are:

- **Go**. The programming language for both master and worker components
- **Docker SDK**. For managing containers on worker nodes
- **HashiCorp Raft**. For leader election and state replication between master nodes
- **UDP multicast**. For heartbeat communication between workers and master
- **JWT (JSON Web Tokens)**. For user authentication

1.3. Thesis Structure

The thesis is organized into five chapters:

- **Chapter 1. Introduction**: presents the context, motivation, objectives, methodology and an overview of the thesis structure.
- **Chapter 2. Theoretical Background and Similar Solutions**: covers the key concepts of

distributed systems (heartbeats, consensus, fault tolerance, load balancing, containerization) and compares existing platforms (Kubernetes, Docker Swarm and HashiCorp Nomad) with the proposed solution. It also defines the expected characteristics of the platform.

- **Chapter 3. Proposed Solution**: describes the architecture of the platform in detail, including the master-worker component design, domain model, heartbeat-based node discovery, fault tolerance mechanisms, load balancing strategies, REST API, service deployment flow, JWT authentication with refresh tokens, role-based access control, audit logging, Raft consensus for state replication and Docker integration.
- **Chapter 4. Testing and Experimental Results**: presents the testing methodology including automated unit tests for the load balancer, scheduler and access control components, as well as manual end-to-end test scenarios covering node discovery, service deployment, fault tolerance, security enforcement and audit logging.
- **Chapter 5. Conclusions**: summarizes the achieved objectives, compares the platform with existing solutions, discusses its limitations and proposes future improvements such as persistent storage, service networking and auto-scaling.

Capitolul 2. Theoretical Background and Similar Solutions

This chapter presents the key concepts behind distributed systems and reviews existing platforms that solve similar problems. Understanding these concepts is important for explaining the design decisions made in the proposed solution.

2.1. Distributed Systems

A distributed system is a collection of independent computers that appear to the user as a single system [1]. The main challenges in distributed systems are:

- **Partial failure**. Some nodes can fail while others continue to work. The system must handle this without stopping entirely.
- **No global clock**. Each node has its own clock and there is no single source of time for the entire system.
- **Network unreliability**. Messages between nodes can be delayed, lost, or delivered out of order.

2.1.1. Node Discovery and Heartbeats

In a cluster, nodes need to know about each other. There are two common approaches:

Static configuration. The list of nodes is defined in a configuration file. Simple, but does not support dynamic scaling.

Heartbeat-based discovery. Each node periodically sends a small message (heartbeat) to announce that it is alive. If a node stops sending heartbeats, it is considered dead. This approach is used by systems like Consul [2] and the proposed platform.

Heartbeats can be sent via TCP (reliable, but more overhead) or UDP (lightweight, no connection setup). UDP multicast allows a single packet to reach multiple receivers at once, which is efficient when multiple master nodes need to receive heartbeats [3].

2.1.2. Consensus and Leader Election

When multiple master nodes need to agree on the state of the cluster, they use a consensus protocol. The most widely used consensus protocol in modern systems is **Raft** [4].

Raft works by electing a leader among the master nodes. The leader processes all write operations and replicates them to the followers. If the leader fails, the followers elect a new leader. Raft guarantees that:

- Only one leader exists at any time.
- All nodes agree on the same sequence of operations.
- The system continues to work as long as a majority of nodes (quorum) are alive. For 3 nodes, the quorum is 2.

2.1.3. Fault Tolerance

Fault tolerance is the ability of a system to continue operating when some of its components fail [1]. Common strategies include **replication** (running multiple copies of a service on different nodes so that if one copy fails, others continue to serve requests), **health monitoring** (periodically checking if nodes are alive via heartbeats and taking action when they are not) and **automatic recovery** (restarting failed services on healthy nodes without human intervention).

2.1.4. Load Balancing

Load balancing distributes work across multiple nodes to avoid overloading any single node. Common strategies include:

- **Round Robin**. Requests are distributed to nodes in a circular order.
- **Least Connections**. The node with the fewest active connections is chosen.
- **Weighted**. Nodes with more available resources receive more work.

The proposed platform implements Least Connections and Weighted strategies with a pluggable interface, allowing new strategies to be added without modifying existing code.

2.1.5. Containerization

Containers are lightweight, isolated environments for running applications. Unlike virtual machines, containers share the host operating system kernel, which makes them faster to start and more efficient in resource usage [5].

Docker is the most widely used container runtime. It provides an API for creating, starting, stopping and removing containers. The proposed platform uses the Docker SDK to manage containers on worker nodes.

2.2. Similar Solutions

Several platforms exist for running containerized services in a cluster. This section compares the most relevant ones with the proposed solution.

2.2.1. Kubernetes

Kubernetes [6] is the industry standard for container orchestration. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF).

Key features:

- Automatic scheduling and scaling of containers
- Self-healing: restarts failed containers, replaces nodes
- Service discovery and load balancing
- Configuration management and secrets
- Extensible through custom resources and operators

Kubernetes uses `etcd` (a distributed key-value store based on Raft) for cluster state and `kubelet` agents on each node for container management.

Comparison with the proposed platform: Kubernetes is a full production platform with hundreds of features. The proposed platform focuses on the core mechanisms (heartbeats, scheduling, fault tolerance, security) and implements them from scratch, providing a clear and simple architecture that is easier to understand and extend. Kubernetes is designed for large-scale production use, while the proposed platform is designed for educational purposes and smaller deployments.

2.2.2. Docker Swarm

Docker Swarm [7] is Docker's built-in orchestration tool. It is simpler than Kubernetes and integrates directly with the Docker CLI.

Key features:

- Uses the standard Docker API
- Built-in service discovery
- Load balancing via ingress routing
- Raft consensus for manager nodes

- TLS encryption for node communication

Comparison with the proposed platform: Docker Swarm is closer in simplicity to the proposed platform. Both use Raft for consensus and Docker for container management. The main differences are that the proposed platform uses UDP multicast for heartbeats (instead of TCP), does not require TLS for internal communication (trusted network assumption) and implements auto-join via heartbeats instead of explicit join tokens.

2.2.3. HashiCorp Nomad

Nomad [8] is a workload orchestrator developed by HashiCorp. Unlike Kubernetes, it can manage not only containers but also standalone applications, Java services and batch jobs.

Key features:

- Single binary, easy to deploy
- Multi-datacenter support
- Integrates with Consul for service discovery and Vault for secrets
- Uses Raft for consensus

Comparison with the proposed platform: Nomad shares the philosophy of simplicity. Both use Raft for consensus and a master-worker architecture. The proposed platform is more focused because it only manages Docker containers and implements all components (discovery, scheduling, security) in a single system, without external dependencies like Consul or Vault.

2.3. Summary of Comparison

Table 2.1 summarizes the key differences between the existing solutions and the proposed platform.

Tabelul 2.1. Comparison of existing solutions with the proposed platform

Feature	Kubernetes	Swarm	Nomad	Proposed
Consensus	etcd (Raft)	Raft	Raft	Raft
Node discovery	kubelet	join token	Consul	heartbeat
Heartbeat protocol	TCP	TCP	TCP	UDP multicast
Container runtime	multiple	Docker	multiple	Docker
Internal encryption	optional	TLS	TLS	none (trusted)
Complexity	very high	medium	medium	low
Load balancing	yes	yes	no (external)	yes
RBAC	yes	limited	yes	yes
Audit log	yes	no	yes	yes

2.4. Expected Characteristics

Based on the analysis of existing solutions and the identified gaps, the proposed platform should meet the following requirements:

- **Automatic node discovery**. Worker nodes should be detected automatically when they start, without manual configuration or explicit join commands.
- **Failure detection within 30 seconds**. The system should detect a dead node within 30 seconds of the last heartbeat and begin container recovery immediately.
- **Container recovery without data loss**. When a node dies, all service parameters (image, environment variables, ports, command) should be preserved in the cluster state, allowing containers to be restarted identically on healthy nodes.

- **Pluggable load balancing**. The load balancing strategy should be interchangeable without modifying the rest of the code.
- **Role-based access control**. The platform should support at least three roles (Admin, Writer, Reader) with clearly separated permissions.
- **Audit trail**. All important actions should be logged in an append-only log that can be queried through the API.
- **High availability**. The master should be replicated across at least 3 nodes using Raft consensus, tolerating the failure of 1 master node.
- **Low complexity**. The platform should be simple enough to understand deploy and extend, unlike production platforms such as Kubernetes.

2.5. Conclusions

The existing platforms are mature and feature-rich, but they come with significant complexity. Kubernetes, in particular, has a steep learning curve, requires extensive infrastructure (etcd cluster, multiple control plane components, networking plugins) and is designed for organizations with dedicated operations teams. Docker Swarm is simpler but has been largely superseded by Kubernetes in the industry. Nomad offers a middle ground but depends on external tools (Consul, Vault) for service discovery and secrets.

The proposed platform fills a gap by implementing the core distributed systems mechanisms in a single, self-contained system. It covers heartbeat-based node discovery using UDP multicast, automatic resource pool allocation, pluggable load balancing with two strategies, fault tolerance with container recovery and split-brain prevention, JWT-based authentication with refresh tokens, role-based access control with three permission levels and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol.

Unlike the existing solutions, the platform is designed with simplicity as a primary goal. The entire codebase is a single Go module with minimal external dependencies. This makes it suitable for educational use, where understanding the underlying concepts is more important than production readiness, and for smaller deployments where Kubernetes would introduce unnecessary complexity.

Capitolul 3. Proposed Solution

The problem addressed by this thesis is running containerized services reliably across a cluster of machines. The key challenges are: discovering nodes automatically, distributing workloads efficiently, recovering from failures without downtime and controlling access to the platform. Existing solutions like Kubernetes solve these problems but are too complex for smaller deployments and educational use.

The proposed solution is a distributed platform built from scratch in Go that implements the core mechanisms needed to manage a cluster: heartbeat-based node discovery, resource pools, pluggable load balancing, automatic fault recovery, JWT-based security with role-based access control and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol.

This chapter describes the architecture in detail. The platform has two main components: the **master node**, which manages the cluster and the **worker nodes**, which run the containers.

3.1. Technology Choices

The platform is implemented in **Go**, chosen for its built-in concurrency model (goroutines and channels), efficient networking libraries, single-binary compilation and strong support for systems programming. Both the master and worker are written in Go.

Heartbeat communication uses **UDP multicast**, which is lightweight (no connection setup, no handshake) and allows a single packet to reach all master nodes simultaneously. This is more efficient than TCP for periodic status messages that do not require acknowledgment.

Docker is used as the container runtime on worker nodes. The Docker SDK for Go provides a simple API for creating, starting and stopping containers programmatically.

For consensus and state replication between master nodes, the platform uses **HashiCorp Raft**, an open-source Go implementation of the Raft consensus protocol.

User authentication is handled through **JWT (JSON Web Tokens)**. JWTs are stateless (no server-side session storage needed) and can encode the user's role directly in the token.

3.2. Component Overview

The platform follows a master-worker architecture. The master node handles all management tasks, while worker nodes execute containers and report their status.

In this architecture, there is a clear separation of roles. The master never runs containers itself. It only makes decisions: which node should run a service, when to restart a failed container, whether a user has permission to perform an action. The workers do the actual work: they run Docker containers, monitor their own CPU and RAM usage and report back to the master through heartbeats.

This separation makes the system easier to reason about. If something goes wrong with a container, the problem is on a worker. If something goes wrong with scheduling or permissions, the problem is on the master. Each component has a single job, which makes debugging simpler.

Figure 3.1 shows the complete architecture with all modules and their connections.

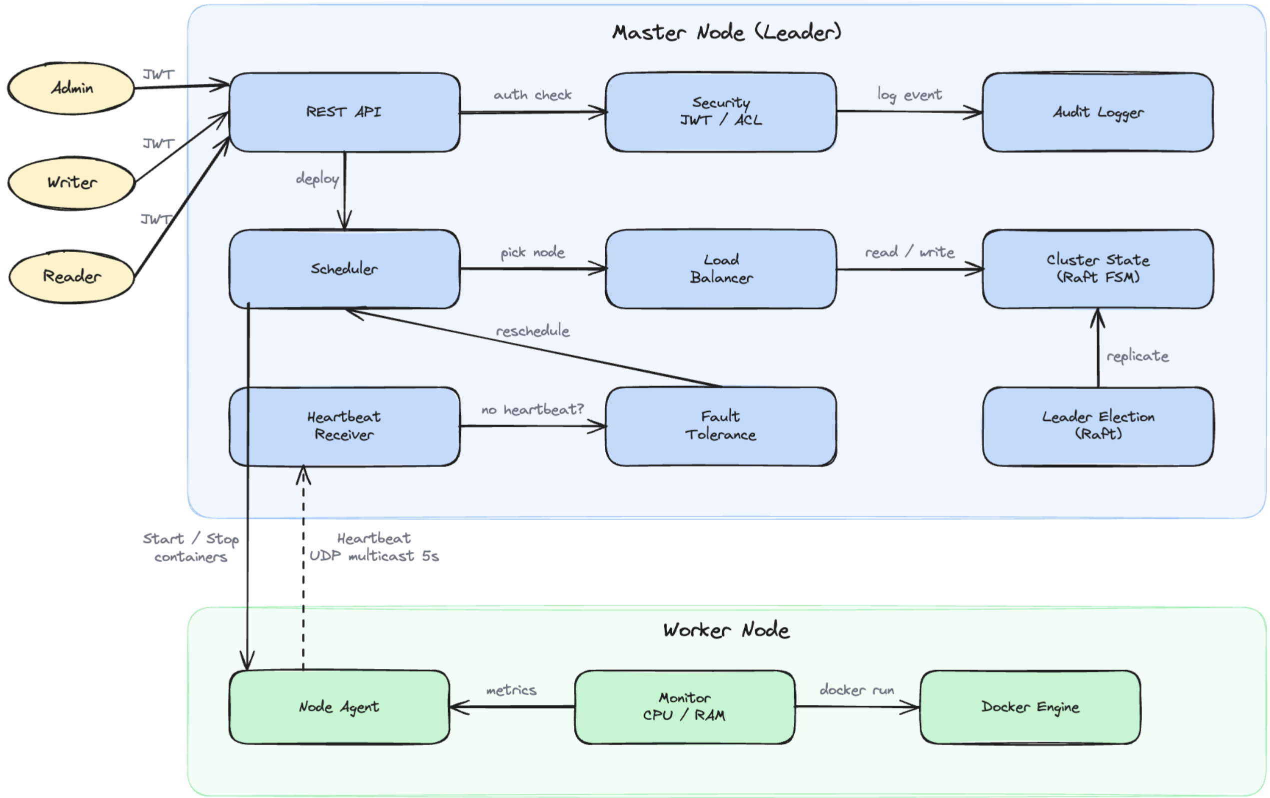


Figura 3.1. Component Overview

3.2.1. Master Node

The master node contains the following modules:

Request pipeline:

- **REST API**. The entry point for users. Exposes endpoints for managing services, nodes and pools.
- **Security**. Handles JWT authentication and ACL authorization. Every request passes through this module before being processed.
- **Audit Logger**. Records all important events in an append-only log.

Scheduling pipeline:

- **Scheduler**. Receives deploy requests and filters nodes in the requested pool that have enough resources. Passes the candidate list to the Load Balancer.
- **Load Balancer**. Selects the best node from the candidate list. Supports two strategies: Least Connections and Weighted.
- **Cluster State (Raft FSM)**. The source of truth for the entire cluster. Stores all data about nodes, services, containers, pools and users. Replicated across 3 master nodes via Raft.

Cluster health:

- **Heartbeat Receiver**. Listens for UDP multicast packets from workers every 5 seconds.
- **Fault Tolerance**. Monitors heartbeats and detects node failures. Marks nodes as SUSPECT after 15 seconds and DEAD after 30 seconds without a heartbeat.
- **Leader Election (Raft)**. Manages leader election across 3 master nodes using the Raft consensus protocol.

3.2.2. Worker Node

Each worker node runs three components:

- **Monitor**. Reads system metrics (CPU usage, RAM usage) from the operating system.
- **Node Agent**. The main process. Sends heartbeats every 5 seconds and listens for commands from the master (StartContainer, KillContainers).
- **Docker Engine**. The agent interacts with Docker to start and stop containers.

The worker is started with the following flags:

- `-id`. Unique node identifier (required)
- `-addr`. TCP address for receiving commands from the master (default: `localhost:7000`)
- `-multicast`. UDP multicast address for heartbeats (default: `239.0.0.1:9090`)
- `-interval`. Heartbeat interval (default: `5s`)

3.2.3. Communication

Master and worker nodes communicate through two channels:

- **Heartbeat**. UDP multicast, every 5 seconds. The worker sends its metrics and the master updates its state. Fire-and-forget, no acknowledgment needed.
- **Commands**. TCP, from master to worker. Used for StartContainer and KillContainers. Only triggered on deploy, delete, or fault recovery.

All nodes run within a trusted internal network, so no TLS encryption is used. Both node-to-master traffic (heartbeats and commands) and Raft traffic between master nodes use plain TCP and UDP. Adding TLS would be a natural next step for an untrusted network.

3.3. Domain Model

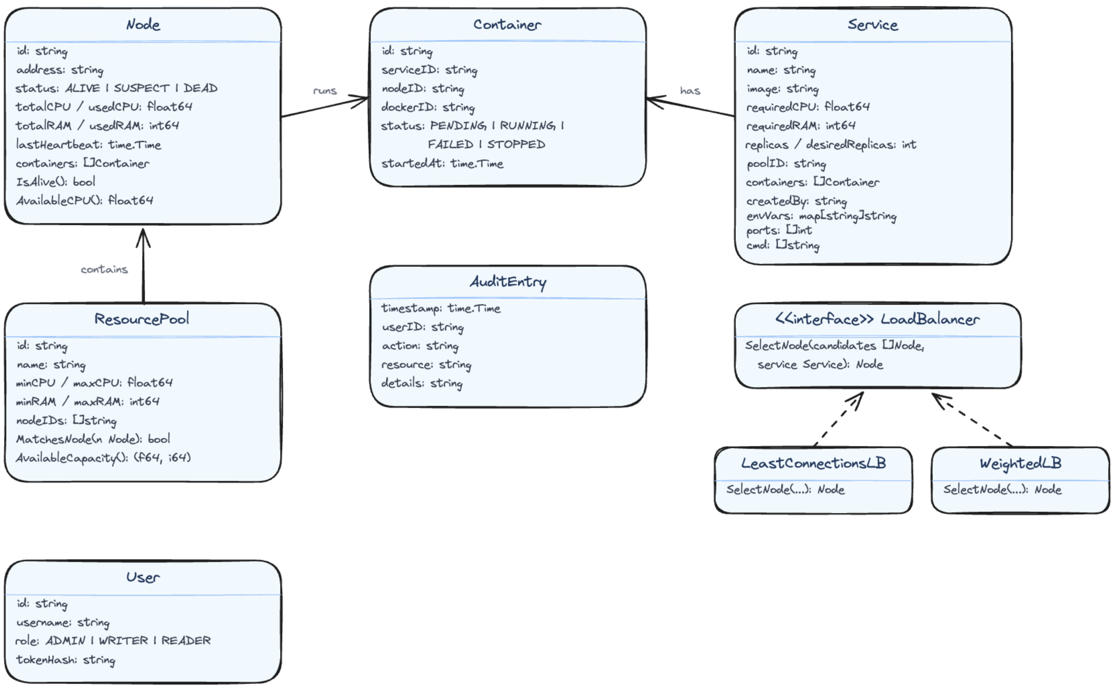


Figura 3.2. Domain Model

The platform uses the following data entities:

Node represents a worker machine in the cluster. It stores the node's address, total and used CPU/RAM, its current status (ALIVE, SUSPECT, DEAD, or REMOVED) and the timestamp of the last received heartbeat.

Service represents what the user wants to run. It contains the Docker image name, required CPU and RAM, the number of desired replicas, the target pool and runtime parameters (environment variables, ports, command). These parameters are saved in the cluster state so that containers can be restarted identically during fault recovery.

Container is a running instance of a service on a specific node. It tracks the Docker container ID and its status (PENDING, SCHEDULED, RUNNING, FAILED, RESCHEDULING, or STOPPED).

ResourcePool groups nodes with similar hardware. Each pool has CPU and RAM ranges. When a new node is discovered, it is automatically assigned to the matching pool based on its resources.

User has an ID, username, role (ADMIN, WRITER, or READER) and a token hash for authentication.

AuditEntry records a single event with a timestamp, user ID, action, resource and details.

LoadBalancer is an interface with a single method: SelectNode. It has two implementations: LeastConnectionsLB (picks the node with fewest active connections) and WeightedLB (picks the node with most available resources).

All these entities are stored in an in-memory data structure called `State`, which acts as the single source of truth for the cluster. The State uses Go maps (one per entity type) protected by a read-write mutex (`sync.RWMutex`).

A **goroutine** is a lightweight thread managed by the Go runtime. Unlike regular operating system threads, goroutines are very cheap to create, so a Go program can run thousands of them at the same time. In this platform, the heartbeat receiver, the fault tolerance checker and the API server each run as a separate goroutine. This means they all execute at the same time, in parallel.

The problem with parallel execution is that two goroutines might try to modify the same data at the same time, which can cause data corruption. A **mutex** (mutual exclusion) solves this problem. It works like a lock on a door: only one goroutine can hold the lock at a time. A `RWMutex` is a special type that allows multiple readers to hold the lock at the same time (since reading does not change anything), but only one writer can hold it at a time. This way, reading the cluster state is fast (no waiting), but writing is safe (no corruption).

3.4. Heartbeat and Node Discovery

The heartbeat mechanism is the foundation of the platform. It solves two problems at once: how does the master know which workers exist and how does it know if a worker is still alive. The answer to both questions is the same: the worker sends a small message (heartbeat) every 5 seconds. If the master receives it, the worker is alive. If the master stops receiving it, the worker is probably dead.

This section describes how heartbeats work, how new nodes are discovered and how nodes are removed from the cluster. Figure 3.3 shows the sequence of messages between a worker, the master and the cluster state.

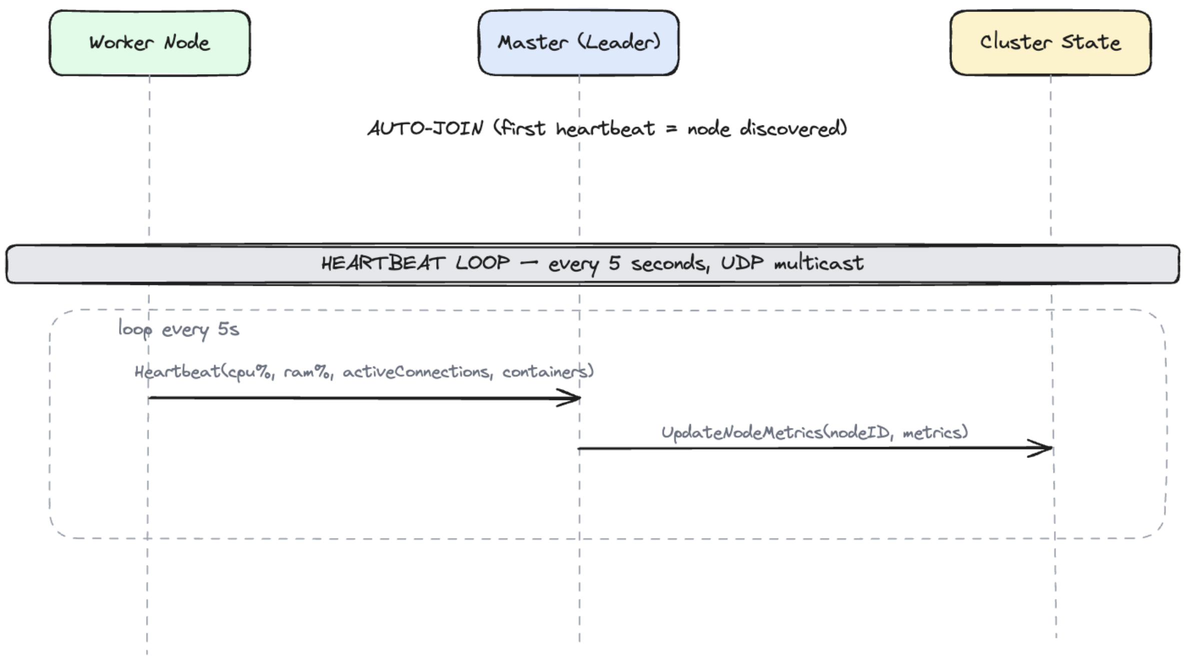


Figura 3.3. Heartbeat and Node Discovery

Node discovery is fully automatic. There is no explicit join or leave request. A worker simply starts sending heartbeats and the master discovers it. When a worker stops, the master notices the missing heartbeats and removes it. Explicit join and leave requests would add unnecessary complexity. The heartbeat-based approach is simpler and handles both planned shutdowns and unexpected crashes in the same way.

3.4.1. Auto-Join

When the master receives the first heartbeat from an unknown node, it automatically adds the node to the cluster state (the in-memory data structure described in section 3.3) with status ALIVE and assigns it to the matching resource pool based on the CPU and RAM values reported in the heartbeat. There is no separate database. The live state is kept in memory using Go maps protected by a mutex, while write operations (services and containers) are also stored in the Raft log on disk. When the master restarts without bootstrapping a fresh cluster, it replays the Raft log to rebuild these entries, and node entries reappear from the next heartbeats.

3.4.2. Heartbeat Loop

Every 5 seconds, each worker sends a UDP multicast packet containing:

- CPU usage (percentage)
- RAM usage (percentage)
- Number of active connections
- List of running containers

The master receives the packet and updates the node's metrics in the cluster state. No response is sent back because the heartbeat is fire-and-forget. The worker does not wait for an answer and does not care if the master received the packet or not. If a packet is lost, the next one will arrive in 5 seconds.

The metrics sent in the heartbeat are important for two reasons. First, the Scheduler uses them to decide where to place new containers. If a node reports 90% CPU usage, the Scheduler will avoid it and pick a node with more free resources. Second, the Fault Tolerance module uses the timestamp of the last heartbeat to detect dead nodes. If no heartbeat arrives for 30 seconds, the node is considered dead.

3.4.3. Node Removal

There is no explicit leave request. When a worker stops (crash, shutdown or network failure), it simply stops sending heartbeats. The master detects the absence through the fault tolerance mechanism (15 seconds SUSPECT, 30 seconds DEAD), marks the node DEAD and moves its containers to other nodes. This means that a planned shutdown and an unexpected crash are handled in exactly the same way. The master does not need to distinguish between the two cases because the result is the same: the node is no longer available and its containers need to be moved elsewhere.

3.5. Fault Tolerance: Detection and Recovery

Fault tolerance is the most complex part of the platform. It handles three scenarios: detecting that a node has failed, restarting the containers that were running on the failed node and preventing duplicate containers when a node that was thought to be dead comes back online. Each scenario requires careful handling to avoid data loss or service interruption.

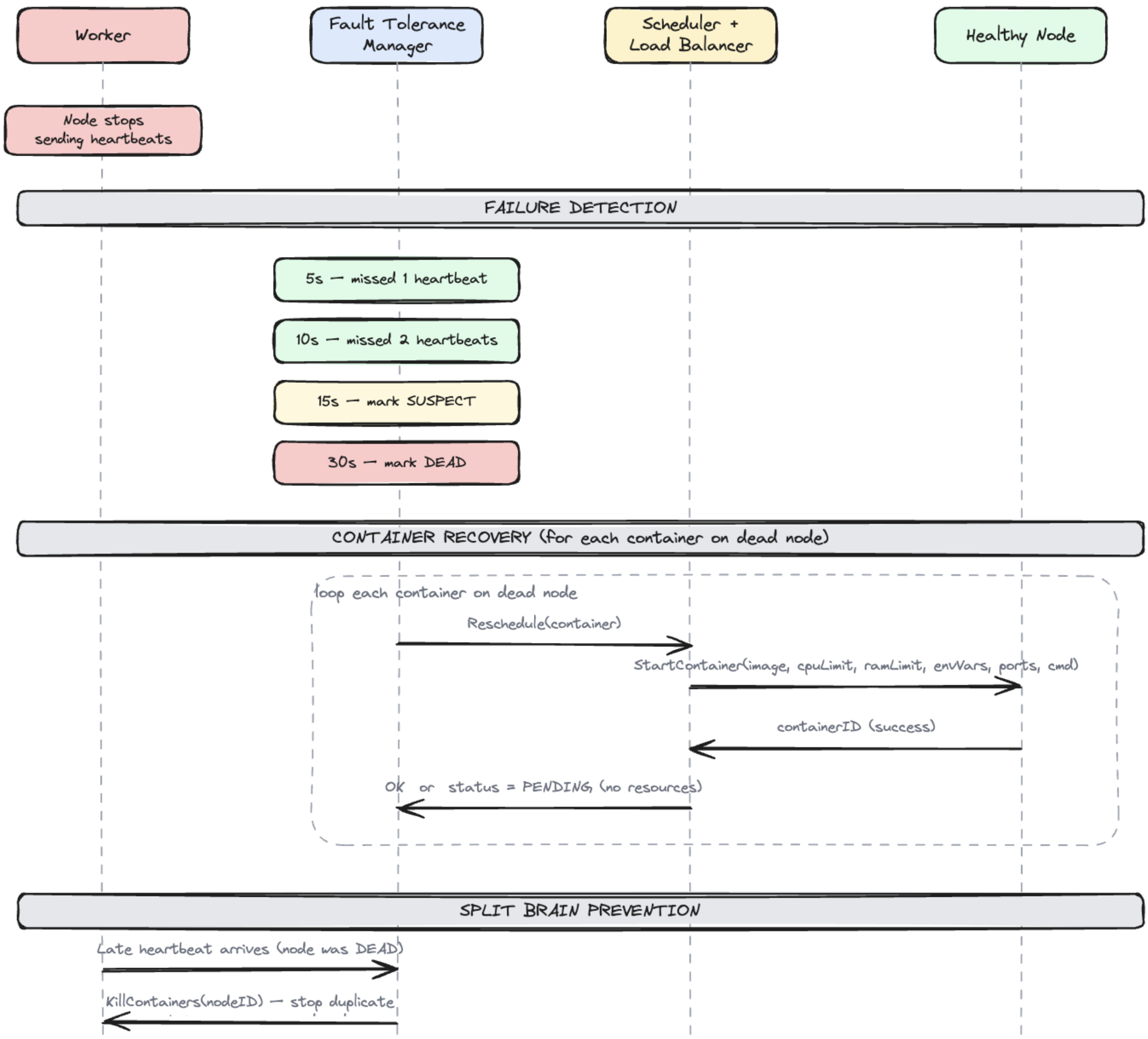


Figura 3.4. Fault Tolerance: Detection and Recovery

3.5.1. Failure Detection

The Fault Tolerance module runs a periodic check (every 5 seconds) on all nodes. For each node, it computes the time since the last heartbeat. After 5 seconds (1 missed heartbeat), no action is taken because it could be network jitter. After 10 seconds (2 missed heartbeats), still no action. After **15 seconds** (3 missed heartbeats), the node is marked **SUSPECT**. Something is probably wrong, but the system gives the node a chance to recover. After **30 seconds** (6 missed heartbeats), the node is marked **DEAD** and recovery begins.

The timing values (15 seconds for SUSPECT and 30 seconds for DEAD) are passed as parameters when creating the Fault Tolerance module. They can be changed without modifying the code. Shorter values make the system react faster to failures but increase the risk of false positives (marking a healthy node as dead because of a temporary network problem). Longer values reduce false positives but mean that containers on a dead node will not be restarted for a longer time.

In the implementation, the check runs as a goroutine with a ticker. Every 5 seconds, the ticker fires and the module loops through all nodes, comparing `time.Now()` with each node's `LastHeartbeat` timestamp. Nodes that are already DEAD or REMOVED are skipped.

3.5.2. Container Recovery

When a node is marked DEAD, its services still need to run. The system starts fresh containers on healthy nodes. For each service that had containers on the dead node, the Fault Tolerance module asks the Scheduler to find a new node. The Scheduler filters nodes in the matching pool with enough resources and passes the candidates to the Load Balancer, which picks the best node. A StartContainer command is then sent to the chosen node with the full service parameters (image, CPU limit, RAM limit, environment variables, ports, command). If no node has enough resources, the container stays in PENDING status until resources become available.

The key point is that containers are not moved from the dead node but started fresh from the service parameters saved in the cluster state. This is why the Service entity stores all runtime parameters (image, environment variables, ports, command). Without them, the system would not know how to recreate the container. The old container on the dead node is marked as STOPPED in the cluster state and a new container entry is created for the replacement.

3.5.3. Split-Brain Prevention

When a node that was marked DEAD comes back online (for example, after a temporary network failure), it sends a heartbeat. The master detects this as a late heartbeat and sends a `KillContainers` command to the recovered node. This stops all containers on the recovered node, because those containers have already been rescheduled onto other healthy nodes. Without this mechanism, the same containers would run on both the recovered node and the new nodes, creating duplicates.

After killing the duplicate containers, the recovered node continues sending heartbeats and becomes available for new scheduling. It does not need to rejoin the cluster or go through any special process. From the master's perspective, it is just a regular ALIVE node with no containers, ready to accept new work.

The term "split-brain" comes from a scenario where the network is split into two parts and each part thinks the other is dead. In this platform, split-brain can happen when the master cannot reach a worker (and marks it DEAD) but the worker is still running its containers. When the network heals, there are two copies of each container. The KillContainers command resolves this by stopping the old copies on the recovered node.

3.6. Worker Node State Machine

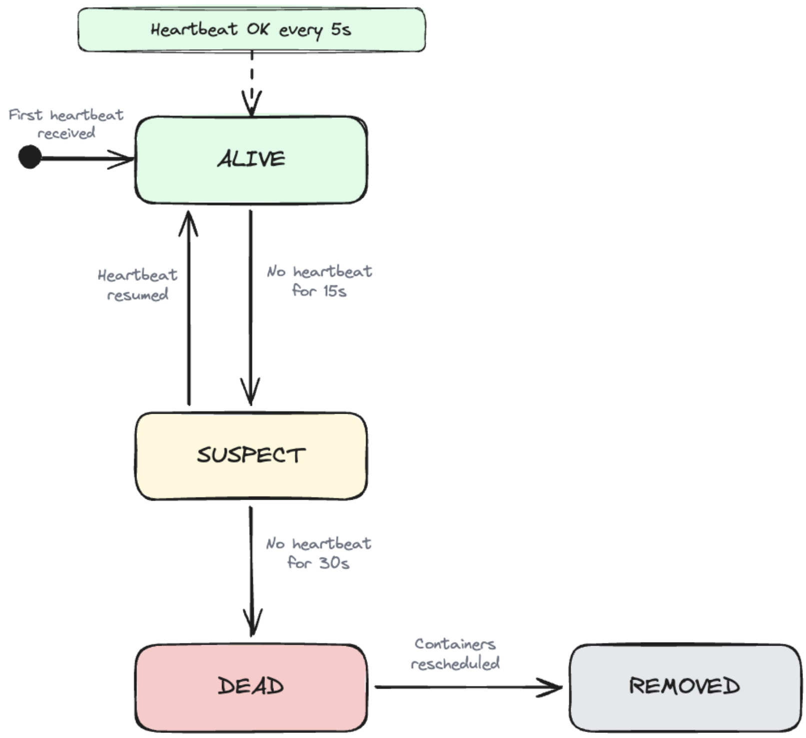


Figura 3.5. Worker Node State Machine

A worker node can be in one of four states:

- **ALIVE**. The initial and normal state. A node enters this state when the master receives its first heartbeat. It sends heartbeats every 5 seconds, runs containers and is available for scheduling.
- **SUSPECT**. The master has not received a heartbeat for 15 seconds. The node might be temporarily unreachable. No action is taken yet.
- **DEAD**. No heartbeat for 30 seconds. The master starts fresh containers on other nodes using the original service parameters.
- **REMOVED**. A reserved final state for a node that has been fully taken out of the cluster. In the current implementation a dead node stays in the DEAD state after its containers are rescheduled; automatic cleanup to REMOVED is left as future work.

The transitions between states are: when the master receives the first heartbeat from a node, it enters the ALIVE state (auto-join). While heartbeats arrive every 5 seconds, the node stays ALIVE. If no heartbeat is received for 15 seconds, the node moves to SUSPECT. If a heartbeat arrives while the node is SUSPECT, it goes back to ALIVE. If no heartbeat is received for 30 seconds, the node moves to DEAD and its containers are rescheduled to other nodes. The dead node stays in the cluster state in the DEAD state; if it later starts sending heartbeats again, it is brought back to ALIVE after its old containers are killed (see split-brain prevention).

The SUSPECT state exists to avoid false positives. A single missed heartbeat could be caused by a short network delay, not a real failure. By waiting for 3 missed heartbeats (15 seconds) before acting, the system avoids unnecessary container restarts that would waste resources and cause brief service interruptions.

3.7. Load Balancer

The Load Balancer selects the best node from a list of candidates. It is defined as a Go interface with a single method:

```
type LoadBalancer interface {
    SelectNode(candidates []*Node, service *Service) *Node
}
```

This design allows the load balancing strategy to be swapped without modifying the rest of the code. The platform provides two implementations:

LeastConnectionsLB picks the node with the fewest running containers. It iterates through the candidate list and selects the node with the smallest container count. This strategy works well when all containers have similar resource requirements.

WeightedLB picks the node with the most available resources. The score is computed as a weighted score between available CPU and available RAM. The node with the highest score is selected. This strategy is better when containers have varying resource needs.

The master can be started with either strategy using the `-lb` flag: `-lb leastconn` (default) or `-lb weighted`.

The `LoadBalancer` interface serves as a public API that allows anyone to implement their own node selection policy. A custom implementation only needs to satisfy the `SelectNode` method and it can be plugged into the platform without modifying any other component.

For example, someone could write a `RandomLB` that picks a random node from the candidate list, or a `LocalityLB` that prefers nodes in the same network zone. The Scheduler does not know or care which strategy is used. It just calls `SelectNode` and gets back a node. This is a common design pattern in Go called an interface, which allows different implementations to be used interchangeably.

The Weighted strategy normalizes both CPU and RAM to a 0 to 1 range (percentage of total resources) before adding them together. This ensures that a node with 8 free CPU cores and 16 GB free RAM is not unfairly favored over a node with 4 free CPU cores and 64 GB free RAM. Both resources contribute equally to the final score.

3.8. REST API

The master exposes a REST API on port 8090 (configurable via `-api` flag). The master also accepts `-multicast` to configure the UDP multicast address for heartbeats (default: `239.0.0.1:9090`). The following endpoints are available:

- `POST /login`. Authenticate with username/password, returns access and refresh tokens
- `POST /refresh`. Exchange a refresh token for a new token pair
- `GET /nodes`. Returns all nodes in the cluster with their status and metrics
- `GET /pools`. Returns all resource pools
- `GET /services`. Returns all deployed services
- `POST /services`. Deploys a new service (triggers scheduling)
- `DELETE /services/{id}`. Deletes a service and stops its containers
- `GET /audit`. Returns audit log entries, with optional filters `?action=...` and `?user=...`

The `POST /services` endpoint accepts a JSON body:

```
{
  "name": "my-app",
  "image": "nginx:latest",
  "cpu": 0.5,
  "ram": 256,
  "replicas": 2,
  "pool": "high-cpu",
  "envVars": {"ENV": "production"},
  "ports": [80],
  "cmd": ["nginx", "-g", "daemon off;"]
}
```

3.9. Service Deployment Flow

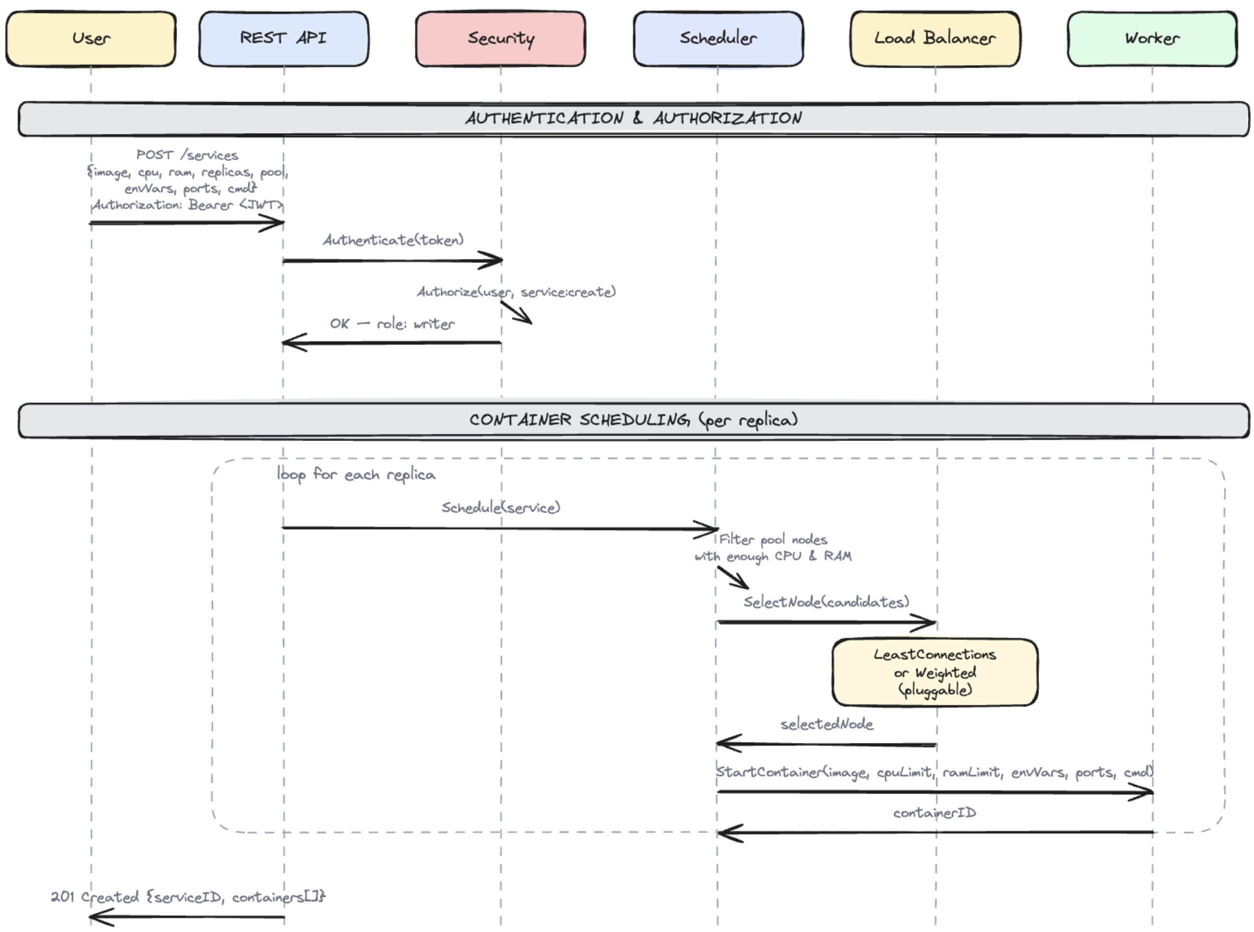


Figura 3.6. Service Deployment Flow

When a user sends a `POST /services` request, the API creates a Service in the cluster state and schedules the requested number of replicas. For each replica, the Scheduler and Load Balancer work together:

1. The Scheduler retrieves all nodes in the requested resource pool.
2. It filters out nodes that are not ALIVE or do not have enough CPU and RAM.

3. It passes the candidate list to the Load Balancer.
4. The Load Balancer returns the best node.
5. A `StartContainer` command is sent to the selected node via TCP with the full service parameters.
6. The worker pulls the Docker image, creates and starts the container and returns the container ID.
7. If no node has enough resources, the container is created with PENDING status.

The API returns a `201 Created` response with the service ID and a list of containers with their statuses and assigned nodes.

This same scheduling flow is used for fault recovery when containers need to be rescheduled after a node dies. The only difference is that during fault recovery, the Fault Tolerance module triggers the scheduling instead of the API. The Scheduler and Load Balancer do not know where the request came from. They just find the best node for the service.

If a service has 3 replicas, the scheduling loop runs 3 times. Each time, it may select a different node because the Load Balancer sees the updated container counts after each placement. For example, with LeastConnections: the first replica goes to worker-1 (0 containers), the second goes to worker-2 (0 containers) and the third goes back to worker-1 (1 container, but worker-2 now also has 1). This naturally distributes replicas across nodes.

When a service is deleted via `DELETE /services/{id}`, the API sends `KillContainers` commands to all workers running that service's containers, marks the containers as STOPPED and removes the service from the cluster state. The KillContainers command includes the service ID so that the worker only stops containers belonging to that service and not all containers on the node.

3.10. Agent: Worker Command Server

Each worker runs a TCP command server that listens for commands from the master. The server accepts two types of commands:

StartContainer. The master sends a JSON command containing all service parameters:

```
{
  "type": "StartContainer",
  "serviceID": "svc-abc123",
  "image": "nginx:latest",
  "cpuLimit": 0.5,
  "ramLimit": 256,
  "envVars": {"ENV": "production"},
  "ports": [80],
  "cmd": ["nginx", "-g", "daemon off;"]
}
```

The worker pulls the Docker image (if not already present), creates a container with the specified resource limits (CPU via `NanoCPUs`, RAM via `Memory`), environment variables, port bindings and command then starts it. The worker responds with:

```
{"success": true, "containerID": "docker-container-id"}
```

KillContainers. The master sends this command during split-brain prevention or when deleting a service. If a `serviceID` is specified, only containers for that service are stopped. If empty, all containers on the worker are stopped.

```
{"type": "KillContainers", "serviceID": "svc-abc123"}
```

Commands are sent as JSON over TCP. The master uses the node's address (reported in heartbeats) to establish a connection, sends the command and waits for a response. This communication is synchronous, meaning the master waits for the worker to confirm before proceeding. If the connection fails (for example because the worker crashed), the master marks the container as PENDING and tries again later.

The worker integrates with Docker through the Docker SDK for Go. A `DockerManager` component wraps the Docker client and tracks all containers started by this worker. When the worker shuts down, it stops and removes all its containers automatically. The list of running container IDs is included in each heartbeat. The master keeps its own record of container placement from the scheduling decisions it makes, so this list is currently informational.

When starting a container, the worker first pulls the Docker image from a registry (like Docker Hub). If the image is already present on the machine, this step is skipped. Then it creates the container with the resource limits (CPU and RAM) that the master specified. The CPU limit is set using Docker's `NanoCPUs` field (0.5 CPU = 500,000,000 nanoseconds of CPU time per second). The RAM limit is set using the `Memory` field in bytes. These limits are enforced by the Linux kernel through a feature called cgroups, which prevents a container from using more resources than it was assigned.

3.11. Container State Machine

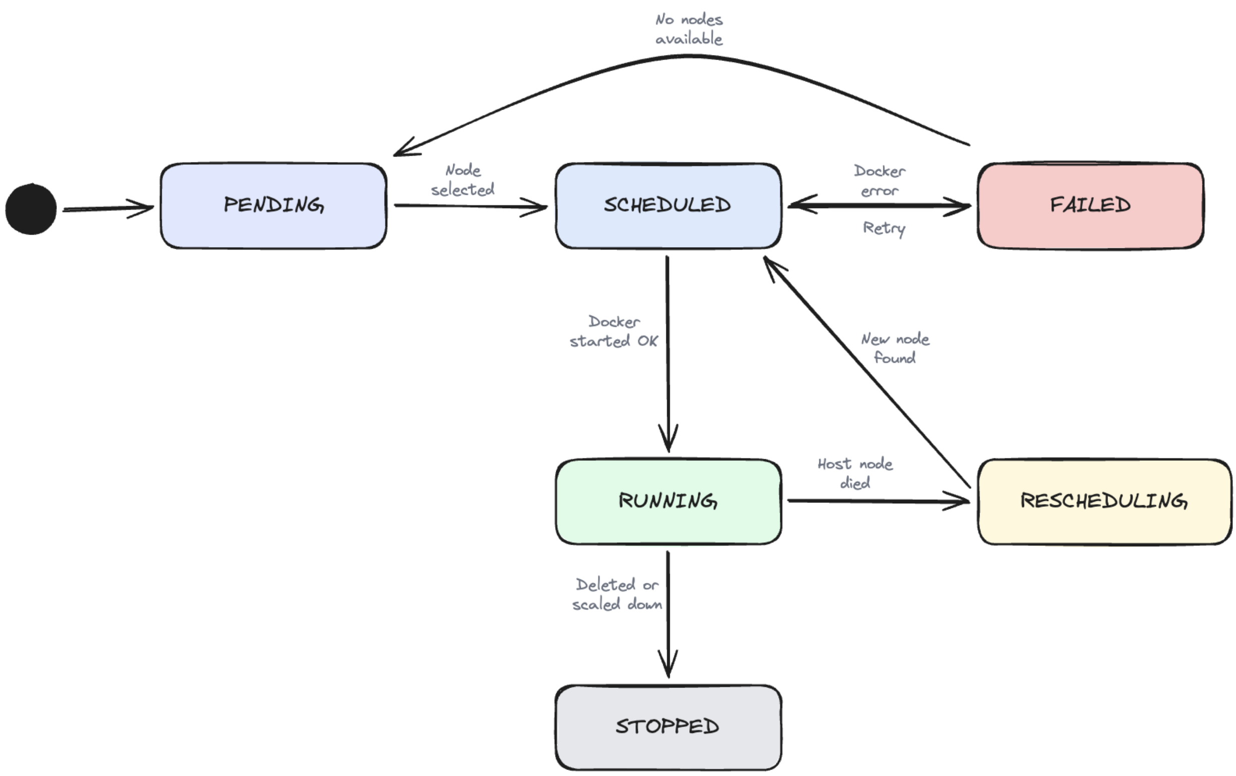


Figura 3.7. Container State Machine

A container goes through the following states:

- **PENDING**. The container needs to run, but no node currently has enough resources. It is created in the PENDING state.
- **SCHEDULED**. A node was found and the container has been assigned. Docker startup is in progress.
- **RUNNING**. Docker started the container successfully. This is the normal operating state.

- **FAILED**. Docker returned an error (invalid image, port conflict, out of memory). The container is marked FAILED.
- **RESCHEDULING**. The host node died. A fresh container needs to be started on another node.
- **STOPPED**. The container was stopped intentionally when its service was deleted, or replaced during fault recovery. This is a final state.

3.12. Security and Audit Architecture

Figura 3.8. Security and Audit Architecture

3.12.1. JWT Authentication with Refresh Tokens

Users authenticate by sending their username and password to the `POST /login` endpoint. The API verifies the credentials against a **SHA-256 hash** stored in the cluster state. SHA-256 is a one-way function: it turns a password like "admin" into a fixed-length string of characters (a hash) like `8c6976e5...`. The important property is that you cannot reverse it, so given the hash you cannot get back the original password. The platform never stores the actual password, only its hash. At login, it hashes the password sent by the user and compares it with the stored hash.

If the credentials are correct, the API returns a token pair:

- **Access token**. A short-lived JWT (15 minutes) containing the user ID, role and expiration. Used in the `Authorization: Bearer <token>` header on every request.
- **Refresh token**. A longer-lived JWT (24 hours) containing only the user ID. Used to obtain a new access token via `POST /refresh` without re-entering credentials.

A **JWT (JSON Web Token)** is a string made of three parts separated by dots: a header, a payload and a signature. The payload contains the user's data (ID, role, expiration time). The signature is created using a secret key known only to the server. When the server receives a JWT, it recalculates the signature and checks if it matches. If someone changed the payload (for example, changed the role from READER to ADMIN), the signature will not match and the token will be rejected.

JWT is **stateless**, which means the server does not need to store any session data. Everything the server needs to know about the user (who they are, what role they have, when the token expires) is inside the token itself. This is different from traditional session-based authentication, where the server stores a session ID in a database and looks it up on every request.

When the access token expires after 15 minutes, the client sends the refresh token to `POST /refresh` and receives a new token pair. This way, the user does not need to enter their password again for 24 hours (the lifetime of the refresh token).

The reason for having two tokens instead of one is security. The access token is sent with every API request, so it is more likely to be intercepted. By keeping it short-lived (15 minutes), the damage from a stolen token is limited. The refresh token is sent only once every 15 minutes to get a new access token, so it is less exposed. If the refresh token is stolen, it can be used for up to 24 hours, but it cannot be used to access the API directly.

In this platform, the secret key used to sign tokens is stored in the master's code. In a production system, it would be stored in a configuration file or a secret manager like HashiCorp Vault.

3.12.2. Role-Based Access Control

The platform has three roles with clearly separated permissions. **ADMIN** has the broadest access: it can create and delete services, manage users, and view nodes, pools and audit logs. **WRITER** can deploy and delete services and can view nodes, pools and audit logs but cannot manage users. **READER** has read-only access and can view services, nodes, pools and audit logs but cannot create or delete anything.

Every protected API endpoint passes through an authentication and authorization middleware. First, it extracts the JWT from the `Authorization` header. Then it validates the signature and checks that the token has not expired. Next, it extracts the role from the token claims and checks the ACL table to see if that role has permission for the requested action on the requested resource. If the role does not have permission, the API returns 403 Forbidden and logs an AUTH_DENIED event in the audit log.

The ACL rules are defined in code as a Go map from role name to a list of allowed actions. For example, the ADMIN role has entries like `{Resource: "services", Action: "create"}` and `{Resource: "users", Action: "create"}`, while the READER role only has entries with `Action: "read"`. This makes it easy to see at a glance what each role can and cannot do. Adding a new role or changing permissions requires modifying a single map in the code.

The login and refresh endpoints are public and do not require a token. All other endpoints require a valid access token. If a request is sent without a token or with an expired token, the API returns 401 Unauthorized.

3.13. Leader Election: Raft Consensus

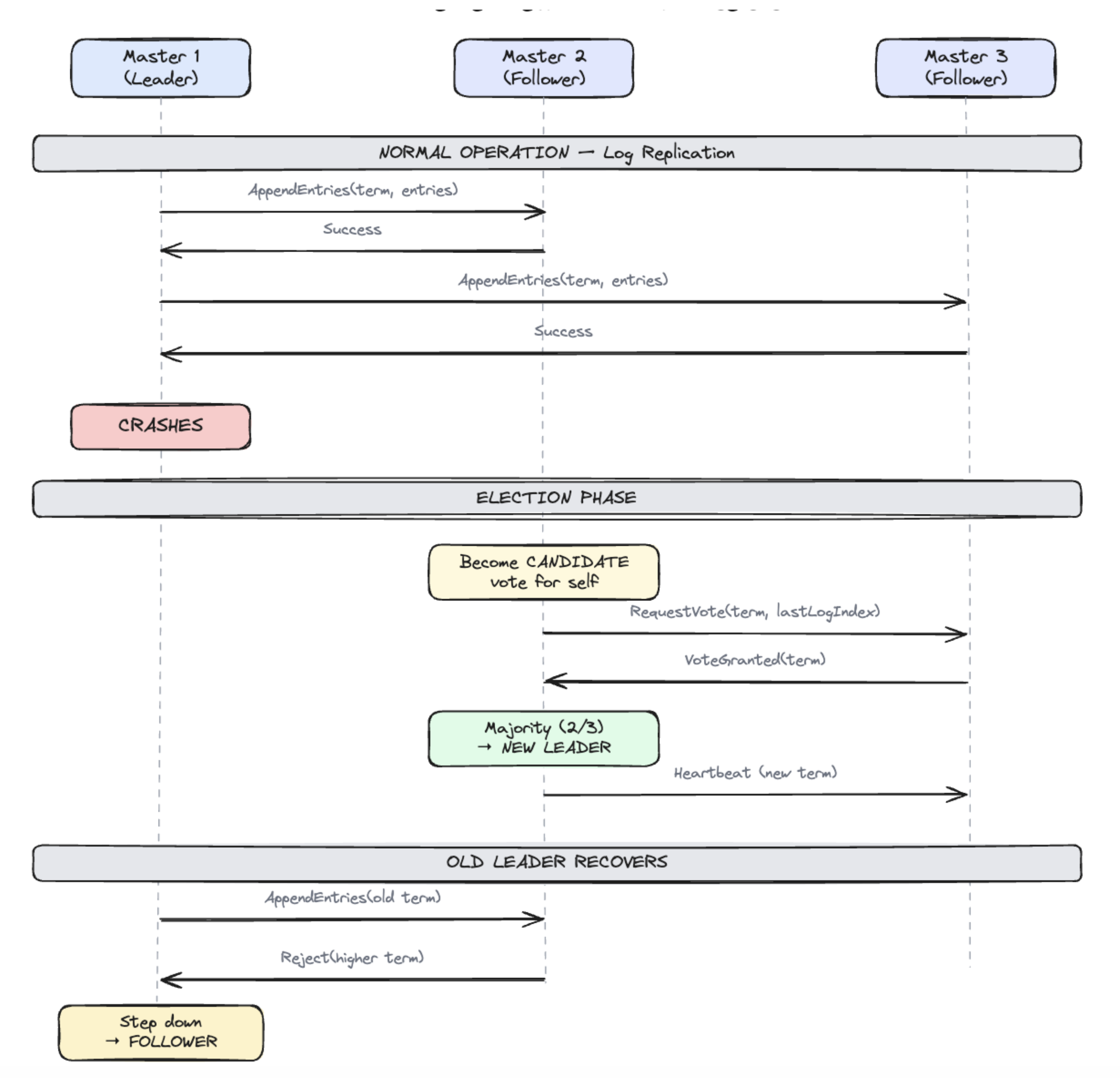


Figura 3.9. Leader Election: Raft Consensus

The platform runs 3 master nodes for high availability. Only one is the leader at any time; the other two are followers that replicate the state.

3.13.1. Normal Operation

The leader sends every state change (node added, service created, container started) to all followers via `AppendEntries` messages. Each follower confirms that it saved the data. This way, every decision is stored on all 3 nodes. The leader waits for a majority of followers to confirm before considering the change committed. With 3 nodes, a majority is 2, so the leader needs only one follower to confirm. This means the cluster can continue working even if one follower is temporarily unreachable.

The followers do not process API write requests and do not make scheduling or recovery decisions; only the leader does. Each master, including the followers, still receives worker heartbeats over UDP multicast and updates its own node list. For replicated data (services and containers), followers only apply what the leader commits through Raft. Their main purpose is to be ready to take over if the leader fails. This keeps all decision-making in one place.

3.13.2. Leader Failure and Election

When the leader crashes, the followers stop receiving Raft heartbeats (these are different from worker heartbeats; Raft heartbeats are sent between master nodes to confirm the leader is alive). Each follower has a random election timeout between 150 and 300 milliseconds. The randomness ensures that not all followers try to become leader at the same time, which would cause a split vote.

When the timeout expires, one follower becomes a candidate. It increments its term number (a counter that goes up with each election), votes for itself and asks the other follower for its vote. The other follower checks if the candidate has up-to-date data (its log is at least as long as the voter's log). If yes, it grants its vote. With 2 votes out of 3 (a majority), the candidate becomes the new leader and starts sending Raft heartbeats to the remaining follower. The entire process takes less than a second.

3.13.3. Old Leader Recovery

When the old leader comes back online, it does not know it has been replaced. It tries to send `AppendEntries` messages with its old term number. The new leader responds with a rejection that includes its higher term number. The old leader sees that its term is outdated, gives up the leader role and becomes a follower. It then syncs its state from the new leader by receiving all the log entries it missed while it was down.

This mechanism guarantees that two leaders can never exist at the same time. The higher term always wins. Even if the old leader comes back before the election is complete, its messages will be rejected because its term is lower.

3.13.4. Implementation

The Raft layer is implemented using the HashiCorp Raft library. Each master node runs a Raft instance with four components. The **Finite State Machine (FSM)** applies committed log entries to the cluster state and supports operations like AddNode, RemoveNode, SetNodeStatus, AddService, RemoveService and SetContainerStatus. The **BoltDB log store** persists the Raft log to disk, ensuring state survives master restarts. The **TCP transport** handles communication between master nodes. A **snapshot mechanism** periodically serializes the cluster state for faster recovery when a new master joins or an existing one restarts.

The first master node is started with the `-raft-bootstrap` flag to initialize the cluster. It must also know about the other masters through the `-raft-peers` flag, which takes a comma-separated list of peer IDs and addresses in the format `id=host:port`. The other masters are started without `-raft-bootstrap` and will automatically join the cluster when the leader contacts them. The master exposes flags for configuration: `-raft-addr`, `-raft-id`, `-raft-dir`, `-raft-bootstrap` and `-raft-peers`.

Each master stores its Raft data in a separate directory (`-raft-dir`), which contains the BoltDB log file (`raft.db`) and periodic snapshots. When a master restarts, Raft reads the log from disk and replays it through the FSM to reconstruct the cluster state.

State changes that go through Raft (AddService, RemoveService, AddContainer) are written to the cluster state only by the FSM, so the API does not modify the state directly. This ensures that all masters apply the same operations in the same order. Heartbeats and node metrics do not go through Raft because they are ephemeral and each master receives them independently via UDP multicast and updates its local state every 5 seconds.

3.13.5. Cluster Deployment Topology

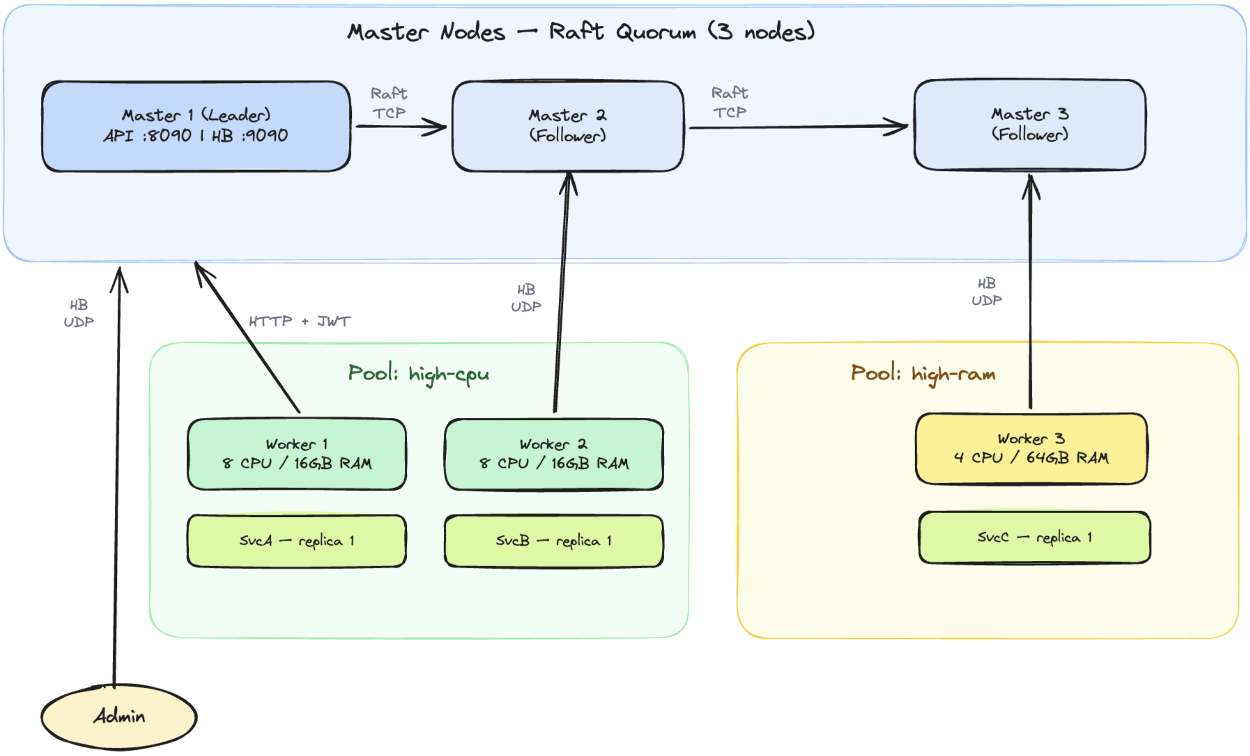


Figura 3.10. Cluster Deployment Topology

The cluster consists of 3 master nodes forming a Raft quorum. Master 1 is the leader, exposing the REST API on port 8090 and the heartbeat receiver on UDP port 9090. Masters 2 and 3 are followers that replicate state via Raft. Worker nodes are organized in resource pools and send heartbeats via UDP multicast to the master. Users connect to the master's REST API using JWT authentication.

3.13.6. Audit Log

All important actions are recorded in an append-only audit log stored in the in-memory cluster state on each master. The log is not replicated through Raft, so each master keeps its own record of the events it observes. The following events are tracked:

- `NODE_DISCOVERED`. When a new node is detected via its first heartbeat
- `NODE_DEAD`. When a node is marked DEAD after 30 seconds without heartbeat
- `SERVICE_CREATE`. When a new service is deployed
- `SERVICE_DELETE`. When a service is deleted
- `CONTAINER_RESCHEDULED`. When a container is moved to another node during fault recovery
- `AUTH_DENIED`. When a user attempts an action they do not have permission for
- `LOGIN`. When a user authenticates successfully

The audit log can be queried through the `GET /audit` endpoint with optional filters for event type (`?action=...`) and user (`?user=...`).

3.14. Implementation

The platform is structured as a single Go module with two binaries: `master` and `worker`. The following packages have been implemented:

- `internal/domain`. Data models (Node, Service, Container, ResourcePool, User, AuditEntry, Heartbeat)
- `internal/cluster`. In-memory cluster state with thread-safe CRUD operations, Raft FSM for state replication, Raft node setup with BoltDB log store and TCP transport
- `internal/heartbeat`. UDP multicast sender (worker) and receiver (master) with auto-join on first heartbeat
- `internal/fault`. Fault tolerance: periodic health checks, SUSPECT/DEAD detection, KillContainers for split-brain prevention
- `internal/loadbalancer`. LoadBalancer interface with LeastConnections and Weighted implementations
- `internal/scheduler`. Filters nodes by pool and resources, delegates to Load Balancer for node selection
- `internal/api`. REST API server with JWT authentication middleware, endpoints for login, refresh, nodes, pools, services
- `internal/security`. JWT token generation/validation (access + refresh tokens), ACL permission checks
- `internal/audit`. Append-only audit logger, integrated with API, heartbeat receiver and fault tolerance
- `internal/agent`. TCP command server (StartContainer, KillContainers), Docker integration via Docker SDK (pull, create, start, stop, remove containers), system metrics monitor (CPU, RAM) and command client for master-to-worker communication

3.15. How to Run

The platform requires Go 1.21+ and Docker to be installed. To build the binaries:

```
go build -o bin/master ./cmd/master
go build -o bin/worker ./cmd/worker
```

To start a cluster with a single master, open three separate terminals:

```
# Terminal 1: start the master
./bin/master -raft-bootstrap

# Terminal 2: start a worker
./bin/worker -id worker-1 -addr localhost:7001

# Terminal 3: start another worker
./bin/worker -id worker-2 -addr localhost:7002
```

To start a cluster with 3 masters for high availability, open five terminals. The first master must know about the other two through the `-raft-peers` flag:

```
# Terminal 1: master-1 (leader, knows about peers)
./bin/master -raft-bootstrap \
  -raft-id master-1 -raft-addr localhost:9001 \
  -raft-dir /tmp/dcp-raft/m1 -api :8090 \
  -raft-peers "master-2=localhost:9002,master-3=localhost:9003"

# Terminal 2: master-2 (follower)
./bin/master -raft-id master-2 \
```

```
  -raft-addr localhost:9002 \
  -raft-dir /tmp/dcp-raft/m2 -api :8091

# Terminal 3: master-3 (follower)
./bin/master -raft-id master-3 \
  -raft-addr localhost:9003 \
  -raft-dir /tmp/dcp-raft/m3 -api :8092

# Terminal 4: worker
./bin/worker -id worker-1 -addr localhost:7001
```

Once all masters are running, master-1 wins the election and becomes leader. If master-1 is stopped, one of the remaining masters will win a new election and become leader within 1 second. The worker continues to send heartbeats to all masters via UDP multicast, so all masters see the same nodes.

The master starts listening for heartbeats on UDP port 9090 and serves the REST API on TCP port 8090. Workers are discovered automatically within 5 seconds. To deploy a service:

```
# Login
curl -X POST http://localhost:8090/login \
  -d '{"username":"admin","password":"admin"}'

# Deploy (use the accessToken from the login response)
curl -X POST http://localhost:8090/services \
  -H "Authorization: Bearer <TOKEN>" \
  -d '{"name":"nginx","image":"nginx:latest",
       "cpu":0.5,"ram":256,"replicas":1,
       "pool":"high-cpu"}'
```

Capitolul 4. Testing and Experimental Results

This chapter presents the testing methodology and results. The platform was tested at two levels: automated unit tests for individual components and manual end-to-end tests to verify the complete workflow.

The purpose of testing is to verify that each component works correctly on its own (unit tests) and that all components work together as a complete system (manual tests). Unit tests are fast, repeatable and can be run automatically. Manual tests are slower but they test real scenarios with actual Docker containers, network communication and user interaction.

The test environment consists of a single macOS machine (Apple Silicon, 11 CPU cores, 18 GB RAM) running Docker Desktop. The master and workers run as separate processes on the same machine, communicating through localhost.

4.1. Unit Tests

Unit tests verify that individual components work correctly in isolation. Each test creates the needed objects in memory, calls a function and checks that the result is correct. The tests are written in Go using the standard `testing` package and can be run with `go test ./...`. No external services (Docker, network) are needed for unit tests.

4.1.1. Load Balancer Tests

Four tests verify the LeastConnections and Weighted strategies:

- **TestLeastConnections_SelectsNodeWithFewestContainers**. Given 3 nodes with 5, 2 and 8 containers respectively, the load balancer selects the node with 2 containers.
- **TestLeastConnections_EmptyCandidates**. When no candidates are provided, returns nil.
- **TestWeighted_SelectsNodeWithMostResources**. Given 3 nodes with different CPU/RAM usage, the load balancer selects the node with the highest available resources (87% CPU free, 87% RAM free).
- **TestWeighted_EmptyCandidates**. When no candidates are provided, returns nil.

4.1.2. Scheduler Tests

Three tests verify the scheduling logic:

- **TestSchedule_FindsNode**. With one alive node that has enough resources, the scheduler returns it.
- **TestSchedule_NotEnoughResources**. When the service requires 100 CPU cores but the node only has 6 available, the scheduler returns an error.
- **TestSchedule_EmptyPool**. When the pool has no nodes, the scheduler returns an error.

4.1.3. ACL Tests

Four tests verify the role-based access control:

- **TestAdminCanDoEverything**. Admin can create, delete and read services, nodes, users and audit.
- **TestReaderCannotCreate**. Reader cannot create or delete services.
- **TestWriterCannotManageUsers**. Writer cannot create users.
- **TestInvalidRole**. An unknown role has no permissions.

4.1.4. Results

All unit tests pass:

```
$ go test ./internal/loadbalancer/ ./internal/scheduler/ ./internal/security/ -v

--- PASS: TestLeastConnections_SelectsNodeWithFewestContainers
--- PASS: TestLeastConnections_EmptyCandidates
--- PASS: TestWeighted_SelectsNodeWithMostResources
--- PASS: TestWeighted_EmptyCandidates
ok   internal/loadbalancer

--- PASS: TestSchedule_FindsNode
--- PASS: TestSchedule_NotEnoughResources
--- PASS: TestSchedule_EmptyPool
ok   internal/scheduler

--- PASS: TestAdminCanDoEverything
--- PASS: TestReaderCannotCreate
--- PASS: TestWriterCannotManageUsers
--- PASS: TestInvalidRole
ok   internal/security
```

4.2. Manual Testing

The following scenarios were tested manually to verify the end-to-end behavior of the platform. Each test follows the same structure: an objective describing what is being verified, the steps to reproduce the test, the expected result with specific details about what the output should look like and a screenshot showing the actual result.

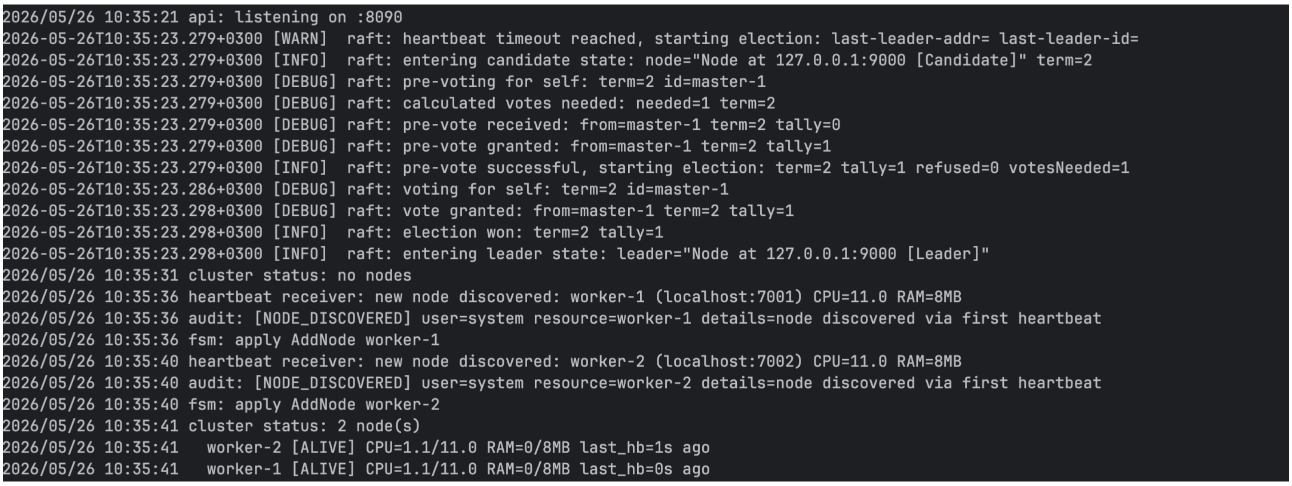
4.2.1. Node Discovery and Heartbeats

Objective: Verify that worker nodes are discovered automatically when they start sending heartbeats and that the master correctly displays their status and metrics.

Steps:

1. Start the master: `./bin/master -raft-bootstrap`
2. Start worker-1: `./bin/worker -id worker-1 -addr localhost:7001`
3. Start worker-2: `./bin/worker -id worker-2 -addr localhost:7002`
4. Wait 10 seconds and observe the master log.

Expected result: The master log shows “new node discovered” for each worker within 5 seconds of starting. The cluster status display (every 10 seconds) lists both workers with status ALIVE, their CPU and RAM usage and the time since their last heartbeat. This confirms that auto-join works and that heartbeats are being received correctly.



```
2026/05/26 10:35:21 api: listening on :8090
2026-05-26T10:35:23.279+0300 [WARN]  raft: heartbeat timeout reached, starting election: last-leader-addr= last-leader-id=
2026-05-26T10:35:23.279+0300 [INFO]  raft: entering candidate state: node="Node at 127.0.0.1:9000 [Candidate]" term=2
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-voting for self: term=2 id=master-1
2026-05-26T10:35:23.279+0300 [DEBUG] raft: calculated votes needed: needed=1 term=2
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-vote received: from=master-1 term=2 tally=0
2026-05-26T10:35:23.279+0300 [DEBUG] raft: pre-vote granted: from=master-1 term=2 tally=1
2026-05-26T10:35:23.279+0300 [INFO]  raft: pre-vote successful, starting election: term=2 tally=1 refused=0 votesNeeded=1
2026-05-26T10:35:23.286+0300 [DEBUG] raft: voting for self: term=2 id=master-1
2026-05-26T10:35:23.298+0300 [DEBUG] raft: vote granted: from=master-1 term=2 tally=1
2026-05-26T10:35:23.298+0300 [INFO]  raft: election won: term=2 tally=1
2026-05-26T10:35:23.298+0300 [INFO]  raft: entering leader state: leader="Node at 127.0.0.1:9000 [Leader]"
2026/05/26 10:35:31 cluster status: no nodes
2026/05/26 10:35:36 heartbeat receiver: new node discovered: worker-1 (localhost:7001) CPU=11.0 RAM=8MB
2026/05/26 10:35:36 audit: [NODE_DISCOVERED] user=system resource=worker-1 details=node discovered via first heartbeat
2026/05/26 10:35:36 fsm: apply AddNode worker-1
2026/05/26 10:35:40 heartbeat receiver: new node discovered: worker-2 (localhost:7002) CPU=11.0 RAM=8MB
2026/05/26 10:35:40 audit: [NODE_DISCOVERED] user=system resource=worker-2 details=node discovered via first heartbeat
2026/05/26 10:35:40 fsm: apply AddNode worker-2
2026/05/26 10:35:41 cluster status: 2 node(s)
2026/05/26 10:35:41   worker-2 [ALIVE] CPU=1.1/11.0 RAM=0/8MB last_hb=1s ago
2026/05/26 10:35:41   worker-1 [ALIVE] CPU=1.1/11.0 RAM=0/8MB last_hb=0s ago
```

Figura 4.1. Node discovery: two workers auto-joined the cluster

4.2.2. Service Deployment

Objective: Verify that a user can authenticate, deploy a service and that the container is actually running on Docker.

Steps:

1. Login as admin: `curl -X POST http://localhost:8090/login -d '{"username":"admin","pass`
2. Deploy nginx using the access token from step 1: `curl -X POST http://localhost:8090/services -H "Authorization: Bearer <TOKEN>" -d '{"name":"nginx","image":"nginx:latest","cpu`
3. Verify the container is running: `docker ps`

Expected result: The login returns a JSON with `accessToken`, `refreshToken` and `expiresIn` (900 seconds). The deploy returns 201 Created with a `serviceID` and a `containers` array where each container has status RUNNING and a `nodeID`. The `docker ps` command shows a container named `dcp-<serviceID>` running on the machine.

```
george@MacBook distributed_cluster_platform %  curl -s -X POST http://localhost:8090/login -d '{"username":"admin","password":"admin"}'
{"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsInJvbGUiOiJBRE1JTiIsImV4cCI6MTc3OTc4NTQ3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.jn-FoiMsuEhkqpoAlqce3j8_ODo-qp2ySA
g5LnkhqyU","refreshToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsImV4cCI6MTc3OTg3MDk3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.5iCbg5_5BRR1ZvBrMIxTiUxsoourryr1RU99loz21E0
","expiresIn":900}

george@MacBook distributed_cluster_platform % curl -s -X POST http://localhost:8090/services -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MS
IsInJvbGUiOiJBRE1JTiIsImV4cCI6MTc3OTc4NTQ3OSwiaWF0IjoxNzc5Nzg0NTc5fQ.jn-FoiMsuEhkqpoAlqce3j8_ODo-qp2ySAg5LnkhqyU" -d '{"name":"nginx","image":"nginx:latest","cpu":0.5,"ram":2
56,"replicas":1,"pool":"high-cpu"}'
{"containers":[{"id":"ctr-dmgavjqt","nodeID":"worker-1","status":"RUNNING"}],"name":"nginx","serviceID":"svc-fquubrsk"}

george@MacBook distributed_cluster_platform % docker ps --filter "name=dcp-"
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS     NAMES
75e752df3322   nginx:latest   "/docker-entrypoint.…"   About a minute ago   Up About a minute   80/tcp    dcp-svc-fquubrsk
```

Figura 4.2. Service deployment: nginx container running on worker-1

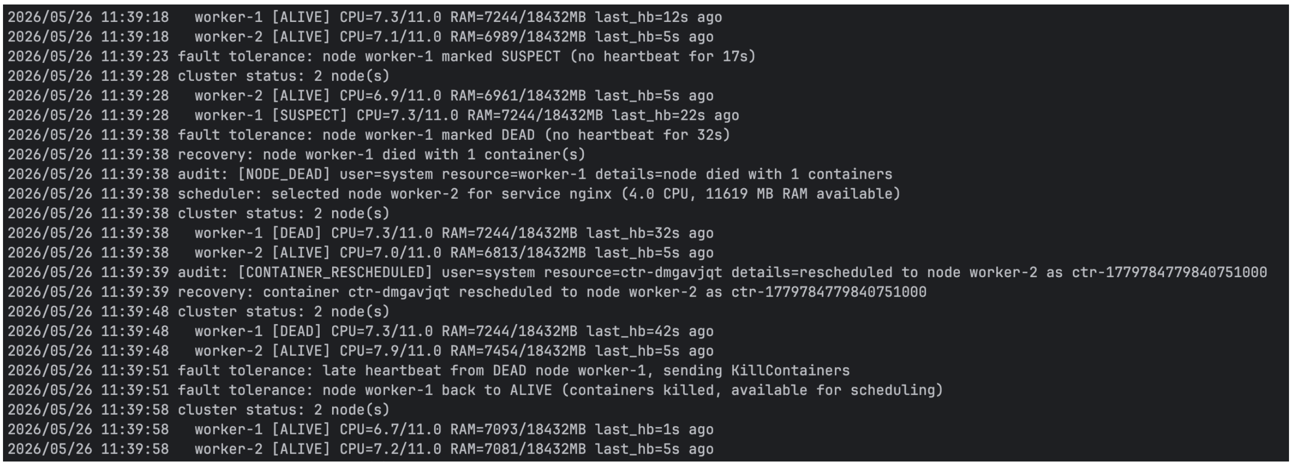
4.2.3. Fault Tolerance

Objective: Verify that the master detects a failed node, transitions it through SUSPECT and DEAD states and handles split-brain when the node recovers.

Steps:

1. With both workers running, stop worker-1 with Ctrl+C.
2. Wait 15 seconds and observe SUSPECT in the master log.
3. Wait 30 seconds and observe DEAD in the master log.
4. Restart worker-1: `./bin/worker -id worker-1 -addr localhost:7001`
5. Observe KillContainers in the master and worker logs.

Expected result: After 15 seconds without heartbeat, the master log shows "node worker-1 marked SUSPECT". After 30 seconds, it shows "node worker-1 marked DEAD". When worker-1 is restarted, the master log shows “late heartbeat from DEAD node worker-1, sending KillContainers” followed by “node worker-1 back to ALIVE”. The worker log confirms it received the KillContainers command. This verifies the complete fault tolerance cycle.



```
2026/05/26 11:39:18   worker-1 [ALIVE] CPU=7.3/11.0 RAM=7244/18432MB last_hb=12s ago
2026/05/26 11:39:18   worker-2 [ALIVE] CPU=7.1/11.0 RAM=6989/18432MB last_hb=5s ago
2026/05/26 11:39:23 fault tolerance: node worker-1 marked SUSPECT (no heartbeat for 17s)
2026/05/26 11:39:28 cluster status: 2 node(s)
2026/05/26 11:39:28   worker-2 [ALIVE] CPU=6.9/11.0 RAM=6961/18432MB last_hb=5s ago
2026/05/26 11:39:28   worker-1 [SUSPECT] CPU=7.3/11.0 RAM=7244/18432MB last_hb=22s ago
2026/05/26 11:39:38 fault tolerance: node worker-1 marked DEAD (no heartbeat for 32s)
2026/05/26 11:39:38 recovery: node worker-1 died with 1 container(s)
2026/05/26 11:39:38 audit: [NODE_DEAD] user=system resource=worker-1 details=node died with 1 containers
2026/05/26 11:39:38 scheduler: selected node worker-2 for service nginx (4.0 CPU, 11619 MB RAM available)
2026/05/26 11:39:38 cluster status: 2 node(s)
2026/05/26 11:39:38   worker-1 [DEAD] CPU=7.3/11.0 RAM=7244/18432MB last_hb=32s ago
2026/05/26 11:39:38   worker-2 [ALIVE] CPU=7.0/11.0 RAM=6813/18432MB last_hb=5s ago
2026/05/26 11:39:39 audit: [CONTAINER_RESCHEDULED] user=system resource=ctr-dmgavjqt details=rescheduled to node worker-2 as ctr-1779784779840751000
2026/05/26 11:39:39 recovery: container ctr-dmgavjqt rescheduled to node worker-2 as ctr-1779784779840751000
2026/05/26 11:39:48 cluster status: 2 node(s)
2026/05/26 11:39:48   worker-1 [DEAD] CPU=7.3/11.0 RAM=7244/18432MB last_hb=42s ago
2026/05/26 11:39:48   worker-2 [ALIVE] CPU=7.9/11.0 RAM=7454/18432MB last_hb=5s ago
2026/05/26 11:39:51 fault tolerance: late heartbeat from DEAD node worker-1, sending KillContainers
2026/05/26 11:39:51 fault tolerance: node worker-1 back to ALIVE (containers killed, available for scheduling)
2026/05/26 11:39:58 cluster status: 2 node(s)
2026/05/26 11:39:58   worker-1 [ALIVE] CPU=6.7/11.0 RAM=7093/18432MB last_hb=1s ago
2026/05/26 11:39:58   worker-2 [ALIVE] CPU=7.2/11.0 RAM=7081/18432MB last_hb=5s ago
```

Figura 4.3. Fault tolerance: node marked SUSPECT, then DEAD, then recovered

4.2.4. Security: Role-Based Access Control

Objective: Verify that the READER role cannot create services while the ADMIN role can.
Steps:

1. Login as reader: `curl -X POST http://localhost:8090/login -d '{"username":"reader","pas`
2. Try to deploy a service using the reader token.
3. Login as admin and deploy the same service.

Expected result: The reader login succeeds but the deploy request returns 403 Forbidden with the error “role READER cannot create services”. The master log shows an AUTH_DENIED audit event. The admin login and deploy succeed with 201 Created. This confirms that the ACL middleware correctly enforces role-based permissions.

```
george@MacBook distributed_cluster_platform %  curl -s -X POST http://localhost:8090/login -d '{"username":"reader","password":"reader"}'
{"accessToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MyIsInJvbGUiOiJSRUFERVIiLCJleHAiOjE3Nzk3ODU3NTksImlhdCI6MTc3OTc4NDg1OX0.
p04Vy4_wi8","refreshToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MyIsImV4cCI6MTc3OTg3MTI1OSwiaWF0IjoxNzc5Nzg0ODU5fQ.NiLAgrPs1
8","expiresIn":900}

george@MacBook distributed_cluster_platform % curl -s -X POST http://localhost:8090/services -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsIn
IsInJvbGUiOiJSRUFERVIiLCJleHAiOjE3Nzk3ODU3NTksImlhdCI6MTc3OTc4NDg1OX0.Kiiuhv-Tjm7DT7vcRgxJqIFtUSNsVI5qep04Vy4_wi8" -d '{"name":"test","image"
licas":1,"pool":"high-cpu"}'

{"error":"role READER cannot create services"}
```

Figura 4.4. Security: reader denied, admin allowed

4.2.5. Audit Log

Objective: Verify that all important actions are recorded in the audit log and that the log can be queried with filters.
Steps:

1. Perform several actions: login, deploy a service, delete a service.
2. Query the full audit log: `curl -H "Authorization: Bearer <TOKEN>" http://localhost:8090/a`
3. Query with a filter: `curl -H "Authorization: Bearer <TOKEN>" http://localhost:8090/audit`

Expected result: The audit log returns a JSON array of events. Each event has a timestamp, user ID, action type (LOGIN, SERVICE_CREATE, NODE_DISCOVERED, AUTH_DENIED, etc.)

and the affected resource. The filtered query returns only events matching the specified action. The log is append-only, so all events from the current session are present in chronological order.

```
george@MacBook distributed_cluster_platform %  curl -s -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJ1MSIsInJvbGUiOiJBRE1JT
NjE5MywiaWF0IjoxNzc5Nzg1MjkzfQ.06Ifkw58VPzrrpqWr30-zrj8bmOA_QJmHMd-SKIhLpA" http://localhost:8090/audit | python3 -m json.tool
[
    {
        "Timestamp": "2026-05-26T11:36:15.666529+03:00",
        "UserID": "system",
        "Action": "NODE_DISCOVERED",
        "Resource": "worker-1",
        "Details": "node discovered via first heartbeat"
    },
    {
        "Timestamp": "2026-05-26T11:36:18.304646+03:00",
        "UserID": "system",
        "Action": "NODE_DISCOVERED",
        "Resource": "worker-2",
        "Details": "node discovered via first heartbeat"
    },
    {
        "Timestamp": "2026-05-26T11:36:19.930504+03:00",
        "UserID": "u1",
        "Action": "LOGIN",
        "Resource": "auth",
        "Details": "user admin logged in"
    },
    {
        "Timestamp": "2026-05-26T11:37:00.772522+03:00",
        "UserID": "",
        "Action": "SERVICE_CREATE",
        "Resource": "svc-fquubrsk",
        "Details": "service nginx created with 1 replicas"
    },
    {
        "Timestamp": "2026-05-26T11:39:38.115845+03:00",
        "UserID": "system",
        "Action": "NODE_DEAD",
        "Resource": "worker-1",
        "Details": "node died with 1 containers"
    },
```

Figura 4.5. Audit log with recorded events

Capitolul 5. Conclusions

This thesis presented the design and implementation of a distributed platform for running containerized services within a cluster. The platform was built from scratch in Go and addresses the core challenges of cluster management: automatic node discovery, resource allocation, load balancing, fault tolerance, security and auditing. The entire codebase is a single Go module with clear package structure, making it easy to read and extend.

5.1. Achieved Objectives

All objectives proposed in the introduction have been achieved:

- **Automatic node discovery**. Worker nodes are discovered automatically via UDP multicast heartbeats. No manual configuration or join tokens are needed. The master detects new nodes within 5 seconds of their first heartbeat.
- **Resource pools**. Nodes are grouped into pools based on CPU and RAM. Services are deployed to specific pools, ensuring they run on appropriate hardware.
- **Load balancing**. Two strategies are implemented (Least Connections and Weighted) behind a pluggable interface that allows custom strategies to be added without modifying existing code.
- **Fault tolerance**. The platform detects node failures within 30 seconds, automatically restarts containers on healthy nodes using the original service parameters and prevents duplicate containers through the KillContainers mechanism.
- **Security**. JWT authentication with refresh tokens and role-based access control (Admin, Writer, Reader) protect all API endpoints.
- **Audit**. All important actions are recorded in an append-only log queryable through the API.
- **High availability**. Raft consensus enables state replication across multiple master nodes with automatic leader election.
- **Docker integration**. Containers are managed through the Docker SDK, supporting image pulling, resource limits, environment variables, ports and custom commands.
- **REST API**. A complete HTTP API allows users to manage services, view nodes and query audit logs.

5.2. Comparison with Similar Solutions

Compared to Kubernetes, Docker Swarm and Nomad (analyzed in Chapter 2), the proposed platform is simpler to understand and deploy. It implements the same fundamental concepts (heartbeats, consensus, scheduling, RBAC) but in a single Go module with minimal dependencies. This makes it suitable for educational purposes and smaller deployments where Kubernetes would be too complex.

The main trade-off is that the platform lacks features expected in production systems: a durable and replicated audit log, service networking, horizontal scaling of masters and support for multiple container runtimes.

5.3. Future Work

Several improvements could extend the platform:

- **Durable audit and full recovery**. Service and container state is already persisted through the Raft log, so a master rebuilds it on restart. The audit log, however, is kept only in memory and is not replicated; persisting and replicating it would be a useful next step.
- **Service networking**. Adding DNS-based service discovery and ingress routing would allow

containers to communicate with each other by name.

- **Auto-scaling**. Automatically adjusting the number of replicas based on CPU or memory usage.
- **Web dashboard**. A graphical interface for monitoring the cluster, deploying services and viewing logs.
- **Multi-runtime support**. Adding support for containerd or Podman in addition to Docker.
- **Health checks**. Allowing services to define custom health check endpoints that the platform monitors.

5.4. Lessons Learned

Building this platform from scratch provided several important lessons about distributed systems. First, handling failures is harder than handling normal operations because most of the complexity comes from edge cases like split-brain, partial failures and race conditions. Second, keeping things simple is a design choice that requires effort. It is easy to add features, but hard to keep the system understandable. Third, UDP multicast is a good fit for heartbeats but requires careful handling since packets can be lost. Fourth, Raft provides strong guarantees but adds complexity to every write operation. Finally, testing distributed systems is difficult because failures are hard to reproduce, so the manual tests were more useful than unit tests for finding real bugs.

Bibliografie

[1] A. S. Tanenbaum and M. V. Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017, ISBN: 978-1-5430-5738-6.

[2] HashiCorp, "Consul documentation," https://developer.hashicorp.com/consul/docs, 2024, Ultima accesare: Mai 2026.

[3] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003, ISBN: 978-0-13-141155-5.

[4] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 2014, pp. 305–319.

[5] Docker Inc., "Docker overview," https://docs.docker.com/get-started/overview/, 2024, Ultima accesare: Mai 2026.

[6] The Kubernetes Authors, "Kubernetes documentation," https://kubernetes.io/docs/, 2024, Ultima accesare: Mai 2026.

[7] Docker Inc., "Swarm mode overview," https://docs.docker.com/engine/swarm/, 2024, Ultima accesare: Mai 2026.

[8] HashiCorp, "Nomad documentation," https://developer.hashicorp.com/nomad/docs, 2024, Ultima accesare: Mai 2026.

Annexes

The following annexes contain the source code of the main components developed for the platform.

Annex 1. Domain Models

```go
package domain

import "time"

// Statuses
const (
  NodeAlive              = "ALIVE"
  NodeSuspect            = "SUSPECT"
  NodeDead               = "DEAD"
  NodeRemoved            = "REMOVED"
  ContainerPending       = "PENDING"
  ContainerScheduled     = "SCHEDULED"
  ContainerRunning       = "RUNNING"
  ContainerFailed        = "FAILED"
  ContainerRescheduling = "RESCHEDULING"
  ContainerStopped       = "STOPPED"
)

// User roles
const (
  RoleAdmin  = "ADMIN"
  RoleWriter = "WRITER"
  RoleReader = "READER"
)

type Node struct {
  ID                string
  Address           string
  Status            string
  TotalCPU          float64
  UsedCPU           float64
  TotalRAM          int64
  UsedRAM           int64
  ActiveConnections int
  LastHeartbeat     time.Time
  Containers        []Container
}

func (n *Node) IsAlive() bool {
  return n.Status == NodeAlive
}

func (n *Node) AvailableCPU() float64 {
  return n.TotalCPU - n.UsedCPU
}

```

```go
func (n *Node) AvailableRAM() int64 {
  return n.TotalRAM - n.UsedRAM
}

type Container struct {
  ID        string
  ServiceID string
  NodeID    string
  DockerID  string
  Status    string
  StartedAt time.Time
}

type Service struct {
  ID              string
  Name            string
  Image           string
  RequiredCPU     float64
  RequiredRAM     int64
  Replicas        int
  DesiredReplicas int
  PoolID          string
  Containers      []Container
  CreatedBy       string
  EnvVars         map[string]string
  Ports           []int
  Cmd             []string
}

type ResourcePool struct {
  ID      string
  Name    string
  MinCPU  float64
  MaxCPU  float64
  MinRAM  int64
  MaxRAM  int64
  NodeIDs []string
}

func (p *ResourcePool) MatchesNode(n Node) bool {
  return n.TotalCPU >= p.MinCPU && n.TotalCPU <= p.MaxCPU &&
    n.TotalRAM >= p.MinRAM && n.TotalRAM <= p.MaxRAM
}

func (p *ResourcePool) AvailableCapacity(nodes []Node) (float64, int64)
 ↪ {
  var cpu float64
  var ram int64
  for _, n := range nodes {
    if n.IsAlive() {
      cpu += n.AvailableCPU()
      ram += n.AvailableRAM()
    }
  }
```

```
	return cpu, ram
}

type AuditEntry struct {
	Timestamp time.Time
	UserID    string
	Action    string
	Resource  string
	Details   string
}

type User struct {
	ID        string
	Username  string
	Role      string
	TokenHash string
}

type Heartbeat struct {
	NodeID            string
	Address           string
	TotalCPU          float64
	UsedCPU           float64
	TotalRAM          int64
	UsedRAM           int64
	ActiveConnections int
	Containers        []string
}
```

Listing 1. internal/domain/models.go – Data models

Annex 2. Cluster State

```go
package cluster

import (
  "fmt"
  "sync"
  "time"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
)

type State struct {
  mu          sync.RWMutex
  nodes       map[string]*domain.Node
  services    map[string]*domain.Service
  containers map[string]*domain.Container
  pools       map[string]*domain.ResourcePool
  users       map[string]*domain.User
  audit       []domain.AuditEntry
}

func NewState() *State {
  return &State{
    nodes:      make(map[string]*domain.Node),
    services:   make(map[string]*domain.Service),
    containers: make(map[string]*domain.Container),
    pools:      make(map[string]*domain.ResourcePool),
    users:      make(map[string]*domain.User),
  }
}

// --- Nodes ---

func (s *State) AddNode(node *domain.Node) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.nodes[node.ID] = node
}

func (s *State) GetNode(id string) (*domain.Node, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  n, ok := s.nodes[id]
  return n, ok
}

func (s *State) GetAllNodes() []*domain.Node {
  s.mu.RLock()
  defer s.mu.RUnlock()
  result := make([]*domain.Node, 0, len(s.nodes))
  for _, n := range s.nodes {
    result = append(result, n)
  }
  return result
```

```
}

func (s *State) RemoveNode(id string) {
  s.mu.Lock()
  defer s.mu.Unlock()
  delete(s.nodes, id)
}

func (s *State) UpdateNodeMetrics(nodeID string, cpuUsed float64,
 ↪ ramUsed int64, activeConns int) error {
  s.mu.Lock()
  defer s.mu.Unlock()
  n, ok := s.nodes[nodeID]
  if !ok {
    return fmt.Errorf("node %s not found", nodeID)
  }
  n.UsedCPU = cpuUsed
  n.UsedRAM = ramUsed
  n.ActiveConnections = activeConns
  n.LastHeartbeat = time.Now()
  n.Status = domain.NodeAlive
  return nil
}

func (s *State) SetNodeStatus(nodeID, status string) error {
  s.mu.Lock()
  defer s.mu.Unlock()
  n, ok := s.nodes[nodeID]
  if !ok {
    return fmt.Errorf("node %s not found", nodeID)
  }
  n.Status = status
  return nil
}

// --- Services ---

func (s *State) AddService(svc *domain.Service) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.services[svc.ID] = svc
}

func (s *State) GetService(id string) (*domain.Service, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  svc, ok := s.services[id]
  return svc, ok
}

func (s *State) GetAllServices() []*domain.Service {
  s.mu.RLock()
  defer s.mu.RUnlock()
  result := make([]*domain.Service, 0, len(s.services))
```

```
  for _, svc := range s.services {
    result = append(result, svc)
  }
  return result
}

func (s *State) RemoveService(id string) {
  s.mu.Lock()
  defer s.mu.Unlock()
  delete(s.services, id)
}

func (s *State) SetServiceReplicas(serviceID string, replicas int)
  ↪ error {
  s.mu.Lock()
  defer s.mu.Unlock()
  svc, ok := s.services[serviceID]
  if !ok {
    return fmt.Errorf("service %s not found", serviceID)
  }
  svc.Replicas = replicas
  return nil
}

// --- Containers ---

func (s *State) AddContainer(c *domain.Container) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.containers[c.ID] = c
}

func (s *State) GetContainer(id string) (*domain.Container, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  c, ok := s.containers[id]
  return c, ok
}

func (s *State) GetContainersByNode(nodeID string) []*domain.Container
  ↪ {
  s.mu.RLock()
  defer s.mu.RUnlock()
  var result []*domain.Container
  for _, c := range s.containers {
    if c.NodeID == nodeID {
      result = append(result, c)
    }
  }
  return result
}

func (s *State) GetActiveContainersByNode(nodeID string)
  ↪ []domain.Container {
```

```go
  s.mu.RLock()
  defer s.mu.RUnlock()
  var result []domain.Container
  for _, c := range s.containers {
    if c.NodeID == nodeID && (c.Status == domain.ContainerRunning || c.Status == domain.ContainerScheduled) {
      result = append(result, *c)
    }
  }
  return result
}

func (s *State) GetContainersByService(serviceID string) []*domain.Container {
  s.mu.RLock()
  defer s.mu.RUnlock()
  var result []*domain.Container
  for _, c := range s.containers {
    if c.ServiceID == serviceID {
      result = append(result, c)
    }
  }
  return result
}

func (s *State) RemoveContainer(id string) {
  s.mu.Lock()
  defer s.mu.Unlock()
  delete(s.containers, id)
}

func (s *State) SetContainerStatus(containerID, status string) error {
  s.mu.Lock()
  defer s.mu.Unlock()
  c, ok := s.containers[containerID]
  if !ok {
    return fmt.Errorf("container %s not found", containerID)
  }
  c.Status = status
  return nil
}

// --- Resource Pools ---

func (s *State) AddPool(pool *domain.ResourcePool) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.pools[pool.ID] = pool
}

func (s *State) GetPool(id string) (*domain.ResourcePool, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  p, ok := s.pools[id]
```

```
  return p, ok
}

func (s *State) GetAllPools() []*domain.ResourcePool {
  s.mu.RLock()
  defer s.mu.RUnlock()
  result := make([]*domain.ResourcePool, 0, len(s.pools))
  for _, p := range s.pools {
    result = append(result, p)
  }
  return result
}

func (s *State) AssignNodeToPool(nodeID string, node *domain.Node) {
  s.mu.Lock()
  defer s.mu.Unlock()
  for _, pool := range s.pools {
    if pool.MatchesNode(*node) {
      for _, existing := range pool.NodeIDs {
        if existing == nodeID {
          return
        }
      }
      pool.NodeIDs = append(pool.NodeIDs, nodeID)
      return
    }
  }
}

func (s *State) GetPoolNodes(poolID string) []*domain.Node {
  s.mu.RLock()
  defer s.mu.RUnlock()
  pool, ok := s.pools[poolID]
  if !ok {
    return nil
  }
  var result []*domain.Node
  for _, nid := range pool.NodeIDs {
    if n, ok := s.nodes[nid]; ok {
      result = append(result, n)
    }
  }
  return result
}

// --- Users ---

func (s *State) AddUser(user *domain.User) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.users[user.ID] = user
}

func (s *State) GetAllUsers() []*domain.User {
```

```
  s.mu.RLock()
  defer s.mu.RUnlock()
  result := make([]*domain.User, 0, len(s.users))
  for _, u := range s.users {
    result = append(result, u)
  }
  return result
}

func (s *State) GetUserByUsername(username string) (*domain.User, bool) {
  s.mu.RLock()
  defer s.mu.RUnlock()
  for _, u := range s.users {
    if u.Username == username {
      return u, true
    }
  }
  return nil, false
}

// --- Audit ---

func (s *State) AppendAudit(entry domain.AuditEntry) {
  s.mu.Lock()
  defer s.mu.Unlock()
  s.audit = append(s.audit, entry)
}

func (s *State) GetAuditLog(filter func(domain.AuditEntry) bool) []domain.AuditEntry {
  s.mu.RLock()
  defer s.mu.RUnlock()
  var result []domain.AuditEntry
  for _, e := range s.audit {
    if filter == nil  filter(e) {
      result = append(result, e)
    }
  }
  return result
}

type Snapshot struct {
  Nodes      []*domain.Node         `json:"nodes"`
  Services   []*domain.Service      `json:"services"`
  Containers []*domain.Container    `json:"containers"`
  Pools      []*domain.ResourcePool `json:"pools"`
  Users      []*domain.User         `json:"users"`
  Audit      []domain.AuditEntry    `json:"audit"`
}

func (s *State) Snapshot() Snapshot {
  s.mu.RLock()
  defer s.mu.RUnlock()
```

```
    snap := Snapshot{Audit: append([]domain.AuditEntry(nil), s.audit...)}
    for _, n := range s.nodes {
      snap.Nodes = append(snap.Nodes, n)
    }
    for _, svc := range s.services {
      snap.Services = append(snap.Services, svc)
    }
    for _, c := range s.containers {
      snap.Containers = append(snap.Containers, c)
    }
    for _, p := range s.pools {
      snap.Pools = append(snap.Pools, p)
    }
    for _, u := range s.users {
      snap.Users = append(snap.Users, u)
    }
    return snap
  }

  func (s *State) Restore(snap Snapshot) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.nodes = make(map[string]*domain.Node, len(snap.Nodes))
    for _, n := range snap.Nodes {
      s.nodes[n.ID] = n
    }
    s.services = make(map[string]*domain.Service, len(snap.Services))
    for _, svc := range snap.Services {
      s.services[svc.ID] = svc
    }
    s.containers = make(map[string]*domain.Container,
     ↪  len(snap.Containers))
    for _, c := range snap.Containers {
      s.containers[c.ID] = c
    }
    s.pools = make(map[string]*domain.ResourcePool, len(snap.Pools))
    for _, p := range snap.Pools {
      s.pools[p.ID] = p
    }
    s.users = make(map[string]*domain.User, len(snap.Users))
    for _, u := range snap.Users {
      s.users[u.ID] = u
    }
    s.audit = append([]domain.AuditEntry(nil), snap.Audit...)
  }
```

Listing 2. internal/cluster/state.go – In-memory cluster state

Annex 3. Heartbeat Receiver

```go
package heartbeat

import (
  "encoding/json"
  "log"
  "net"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
)

type Receiver struct {
  multicastAddr    string
  state            *cluster.State
  onLateHeartbeat  func(nodeID string)
  onNodeDiscovered func(nodeID string)
  stop             chan struct{}
}

func NewReceiver(multicastAddr string, state *cluster.State, onLateHeartbeat func(nodeID string), onNodeDiscovered func(nodeID string)) *Receiver {
  return &Receiver{
    multicastAddr:    multicastAddr,
    state:            state,
    onLateHeartbeat:  onLateHeartbeat,
    onNodeDiscovered: onNodeDiscovered,
    stop:             make(chan struct{}),
  }
}

func (r *Receiver) Start() {
  addr, err := net.ResolveUDPAddr("udp", r.multicastAddr)
  if err != nil {
    log.Fatalf("heartbeat receiver: resolve addr: %v", err)
  }

  conn, err := net.ListenMulticastUDP("udp", nil, addr)
  if err != nil {
    log.Fatalf("heartbeat receiver: listen: %v", err)
  }
  defer conn.Close()

  conn.SetReadBuffer(8192)
  log.Printf("heartbeat receiver: listening on %s", r.multicastAddr)

  buf := make([]byte, 4096)
  for {
    select {
    case <-r.stop:
      log.Println("heartbeat receiver: stopped")
      return
    default:
```

```
         n, _, err := conn.ReadFromUDP(buf)
         if err != nil {
            log.Printf("heartbeat receiver: read: %v", err)
            continue
         }
         r.handleHeartbeat(buf[:n])
      }
   }
}

func (r *Receiver) Stop() {
   close(r.stop)
}

func (r *Receiver) handleHeartbeat(data []byte) {
   var hb domain.Heartbeat
   if err := json.Unmarshal(data, &hb); err != nil {
      log.Printf("heartbeat receiver: unmarshal: %v", err)
      return
   }

   node, exists := r.state.GetNode(hb.NodeID)
   if !exists {
      newNode := &domain.Node{
         ID:        hb.NodeID,
         Address:   hb.Address,
         Status:    domain.NodeAlive,
         TotalCPU: hb.TotalCPU,
         UsedCPU:   hb.UsedCPU,
         TotalRAM: hb.TotalRAM,
         UsedRAM:   hb.UsedRAM,
      }
      r.state.AddNode(newNode)
      r.state.AssignNodeToPool(hb.NodeID, newNode)
      log.Printf("heartbeat receiver: new node discovered: %s (%s) CPU=%.1f RAM=%dMB",
         hb.NodeID, hb.Address, hb.TotalCPU, hb.TotalRAM)
      if r.onNodeDiscovered != nil {
         r.onNodeDiscovered(hb.NodeID)
      }
   } else if node.Status == domain.NodeDead && r.onLateHeartbeat != nil {
      r.onLateHeartbeat(hb.NodeID)
   }

   // Update metrics for every heartbeat
   r.state.UpdateNodeMetrics(hb.NodeID, hb.UsedCPU, hb.UsedRAM, hb.ActiveConnections)
}
```

Listing 3. internal/heartbeat/receiver.go – UDP multicast receiver with auto-join

Annex 4. Fault Tolerance

```go
package fault

import (
  "log"
  "time"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/agent"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
)

type Tolerance struct {
  state         *cluster.State
  checkInterval time.Duration
  suspectAfter  time.Duration
  deadAfter     time.Duration
  onNodeDead    func(nodeID string, containers []*domain.Container)
  stop          chan struct{}
}

func NewTolerance(state *cluster.State, checkInterval time.Duration, suspectAfter time.Duration, deadAfter time.Duration, onNodeDead func(nodeID string, containers []*domain.Container)) *Tolerance {
  return &Tolerance{
    state:         state,
    checkInterval: checkInterval,
    suspectAfter:  suspectAfter,
    deadAfter:     deadAfter,
    onNodeDead:    onNodeDead,
    stop:          make(chan struct{}),
  }
}

func (t *Tolerance) Start() {
  ticker := time.NewTicker(t.checkInterval)
  defer ticker.Stop()

  log.Printf("fault tolerance: started, check every %s, suspect after %s, dead after %s",
    t.checkInterval, t.suspectAfter, t.deadAfter)

  for {
    select {
    case <-ticker.C:
      t.check()
    case <-t.stop:
      log.Println("fault tolerance: stopped")
      return
    }
  }
}

func (t *Tolerance) Stop() {
```

```go
    close(t.stop)
  }

  func (t *Tolerance) check() {
    nodes := t.state.GetAllNodes()
    now := time.Now()

    for _, node := range nodes {
      if node.Status == domain.NodeDead  node.Status == domain.NodeRemoved {
        continue
      }

      since := now.Sub(node.LastHeartbeat)

      switch {
      case since >= t.deadAfter && node.Status != domain.NodeDead:
        // 30s without heartbeat -> DEAD
        log.Printf("fault tolerance: node %s marked DEAD (no heartbeat for %s)", node.ID, since.Round(time.Second))
        t.state.SetNodeStatus(node.ID, domain.NodeDead)

        // Get containers on this node and trigger recovery
        containers := t.state.GetContainersByNode(node.ID)
        if len(containers) > 0 && t.onNodeDead != nil {
          t.onNodeDead(node.ID, containers)
        }

      case since >= t.suspectAfter && node.Status == domain.NodeAlive:
        // 15s without heartbeat -> SUSPECT
        log.Printf("fault tolerance: node %s marked SUSPECT (no heartbeat for %s)", node.ID, since.Round(time.Second))
        t.state.SetNodeStatus(node.ID, domain.NodeSuspect)
      }
    }
  }

  // HandleLateHeartbeat handles a heartbeat from a node that was marked DEAD.
  // Sends KillContainers to stop duplicate containers on the recovered node.
  func (t *Tolerance) HandleLateHeartbeat(nodeID string) {
    node, exists := t.state.GetNode(nodeID)
    if !exists {
      return
    }
    if node.Status != domain.NodeDead {
      return
    }

    log.Printf("fault tolerance: late heartbeat from DEAD node %s, sending KillContainers", nodeID)

    addr := node.Address
```

```go
    go func() {
      if _, err := agent.SendCommand(addr, agent.Command{Type: agent.CmdKillContainers}); err != nil {
        log.Printf("fault tolerance: failed to send KillContainers to %s: %v", nodeID, err)
      }
    }()

    // Mark containers on this node as stopped (they were already rescheduled)
    containers := t.state.GetContainersByNode(nodeID)
    for _, c := range containers {
      t.state.SetContainerStatus(c.ID, domain.ContainerStopped)
    }

    // Node goes back to ALIVE, available for new scheduling
    t.state.SetNodeStatus(nodeID, domain.NodeAlive)
    log.Printf("fault tolerance: node %s back to ALIVE (containers killed, available for scheduling)", nodeID)
  }
```

Listing 4. internal/fault/tolerance.go – Failure detection and recovery

Annex 5. Load Balancer

```go
package loadbalancer

import
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"

// LoadBalancer selects the best node from a list of candidates.
type LoadBalancer interface {
  SelectNode(candidates []*domain.Node, service *domain.Service)
    *domain.Node
}
```

Listing 5. internal/loadbalancer/balancer.go – LoadBalancer interface

```go
package loadbalancer

import
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"

// LeastConnectionsLB picks the node with the fewest running
  containers.
type LeastConnectionsLB struct{}

func (lb *LeastConnectionsLB) SelectNode(candidates []*domain.Node,
  service *domain.Service) *domain.Node {
  if len(candidates) == 0 {
    return nil
  }

  best := candidates[0]
  for _, n := range candidates[1:] {
    if len(n.Containers) < len(best.Containers) {
      best = n
    }
  }
  return best
}
```

Listing 6. internal/loadbalancer/leastconn.go – Least Connections strategy

```go
package loadbalancer

import
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"

// WeightedLB picks the node with the most available resources.
// Score is computed as a weighted score between available CPU and
  available RAM.
type WeightedLB struct{}

func (lb *WeightedLB) SelectNode(candidates []*domain.Node, service
  *domain.Service) *domain.Node {
```

```
  if len(candidates) == 0 {
    return nil
  }

  best := candidates[0]
  bestScore := score(best)
  for _, n := range candidates[1:] {
    s := score(n)
    if s > bestScore {
      best = n
      bestScore = s
    }
  }
  return best
}

// score computes a weighted score between available CPU and available
  ↪ RAM.
// Both are normalized to a 0-1 range (percentage of total) so they
  ↪ contribute equally.
func score(n *domain.Node) float64 {
  var cpuScore, ramScore float64
  if n.TotalCPU > 0 {
    cpuScore = n.AvailableCPU() / n.TotalCPU
  }
  if n.TotalRAM > 0 {
    ramScore = float64(n.AvailableRAM()) / float64(n.TotalRAM)
  }
  return cpuScore + ramScore
}
```

Listing 7. internal/loadbalancer/weighted.go – Weighted strategy

Annex 6. Scheduler

```go
package scheduler

import (
	"fmt"
	"log"

	"github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
	"github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
	"github.com/GeorgeMi/Distributed-Cluster-Platform/internal/loadbalancer"
)

type Scheduler struct {
	state *cluster.State
	lb    loadbalancer.LoadBalancer
}

func NewScheduler(state *cluster.State, lb loadbalancer.LoadBalancer) *Scheduler {
	return &Scheduler{
		state: state,
		lb:    lb,
	}
}

// Schedule finds the best node for a service and returns it.
// It filters nodes in the service's pool that are alive and have enough resources.
func (s *Scheduler) Schedule(service *domain.Service) (*domain.Node, error) {
	poolNodes := s.state.GetPoolNodes(service.PoolID)
	if len(poolNodes) == 0 {
		return nil, fmt.Errorf("no nodes in pool %s", service.PoolID)
	}

	// Filter: alive + enough resources
	var candidates []*domain.Node
	for _, n := range poolNodes {
		if n.IsAlive() && n.AvailableCPU() >= service.RequiredCPU && n.AvailableRAM() >= service.RequiredRAM {
			c := *n
			c.Containers = s.state.GetActiveContainersByNode(n.ID)
			candidates = append(candidates, &c)
		}
	}

	if len(candidates) == 0 {
		return nil, fmt.Errorf("no nodes with enough resources (need %.1f CPU, %d MB RAM)", service.RequiredCPU, service.RequiredRAM)
	}

	node := s.lb.SelectNode(candidates, service)
	if node == nil {
		return nil, fmt.Errorf("load balancer returned no node")
```

```
    }

    log.Printf("scheduler: selected node %s for service %s (%.1f CPU, %d MB RAM available)",
        node.ID, service.Name, node.AvailableCPU(), node.AvailableRAM())

    return node, nil
}
```

Listing 8. internal/scheduler/scheduler.go – Node selection by pool and resources

Annex 7. REST API Server

```
package api

import (
  "crypto/sha256"
  "encoding/json"
  "fmt"
  "log"
  "math/rand"
  "net/http"
  "strings"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/agent"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/audit"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/cluster"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/scheduler"
  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/security"
)

type Server struct {
  state     *cluster.State
  scheduler *scheduler.Scheduler
  jwt       *security.JWTManager
  audit     *audit.Logger
  raftNode  *cluster.RaftNode
  addr      string
}

func NewServer(addr string, state *cluster.State, sched
 ↪  *scheduler.Scheduler, jwt *security.JWTManager, auditLog
 ↪  *audit.Logger, raftNode *cluster.RaftNode) *Server {
  return &Server{
    addr:      addr,
    state:     state,
    scheduler: sched,
    jwt:       jwt,
    audit:     auditLog,
    raftNode:  raftNode,
  }
}

func (s *Server) Start() {
  mux := http.NewServeMux()

  // Public endpoints
  mux.HandleFunc("POST /login", s.login)
  mux.HandleFunc("POST /refresh", s.refresh)

  // Protected endpoints
  mux.HandleFunc("GET /nodes", s.auth("nodes", "read", s.getNodes))
  mux.HandleFunc("GET /pools", s.auth("pools", "read", s.getPools))
  mux.HandleFunc("GET /services", s.auth("services", "read",
   ↪  s.getServices))
```

```
  mux.HandleFunc("POST /services", s.auth("services", "create", s.leaderOnly(s.createService)))
  mux.HandleFunc("DELETE /services/{id}", s.auth("services", "delete", s.leaderOnly(s.deleteService)))
  mux.HandleFunc("GET /audit", s.auth("audit", "read", s.getAudit))

  log.Printf("api: listening on %s", s.addr)
  if err := http.ListenAndServe(s.addr, mux); err != nil {
    log.Fatalf("api: %v", err)
  }
}

// auth is a middleware that checks JWT and ACL permissions.
func (s *Server) auth(resource, action string, next http.HandlerFunc) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    // Extract token from Authorization header
    authHeader := r.Header.Get("Authorization")
    if authHeader == ""  !strings.HasPrefix(authHeader, "Bearer ") {
      writeJSON(w, http.StatusUnauthorized, map[string]string{"error": "missing or invalid Authorization header"})
      return
    }
    tokenStr := strings.TrimPrefix(authHeader, "Bearer ")

    // Validate token
    claims, err := s.jwt.ValidateAccessToken(tokenStr)
    if err != nil {
      writeJSON(w, http.StatusUnauthorized, map[string]string{"error": "invalid token: " + err.Error()})
      return
    }

    // Check ACL
    if !security.CheckPermission(claims.Role, resource, action) {
      s.audit.Log(claims.UserID, audit.EventAuthDenied, resource, fmt.Sprintf("role %s cannot %s %s", claims.Role, action, resource))
      writeJSON(w, http.StatusForbidden, map[string]string{"error": fmt.Sprintf("role %s cannot %s %s", claims.Role, action, resource)})
      return
    }

    next(w, r)
  }
}

func (s *Server) leaderOnly(next http.HandlerFunc) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    if s.raftNode != nil && !s.raftNode.IsLeader() {
      writeJSON(w, http.StatusServiceUnavailable, map[string]string{
        "error":      "not the leader; retry against the current leader",
```

```go
				"leaderRaft": s.raftNode.LeaderAddress(),
			})
			return
		}
		next(w, r)
	}
}

// --- Auth endpoints ---

type LoginRequest struct {
	Username string `json:"username"`
	Password string `json:"password"`
}

// POST /login - authenticate and get tokens
func (s *Server) login(w http.ResponseWriter, r *http.Request) {
	var req LoginRequest
	if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
		writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON"})
		return
	}

	user, ok := s.state.GetUserByUsername(req.Username)
	if !ok {
		writeJSON(w, http.StatusUnauthorized, map[string]string{"error": "invalid credentials"})
		return
	}

	// Check password hash
	hash := fmt.Sprintf("%x", sha256.Sum256([]byte(req.Password)))
	if hash != user.TokenHash {
		writeJSON(w, http.StatusUnauthorized, map[string]string{"error": "invalid credentials"})
		return
	}

	tokens, err := s.jwt.GenerateTokenPair(user.ID, user.Role)
	if err != nil {
		writeJSON(w, http.StatusInternalServerError, map[string]string{"error": "failed to generate token"})
		return
	}

	s.audit.Log(user.ID, audit.EventLogin, "auth", fmt.Sprintf("user %s logged in", user.Username))
	writeJSON(w, http.StatusOK, tokens)
}

type RefreshRequest struct {
	RefreshToken string `json:"refreshToken"`
}
```

```go

// POST /refresh - get new access token using refresh token
func (s *Server) refresh(w http.ResponseWriter, r *http.Request) {
  var req RefreshRequest
  if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    writeJSON(w, http.StatusBadRequest, map[string]string{"error":
      "invalid JSON"})
    return
  }

  userID, err := s.jwt.ValidateRefreshToken(req.RefreshToken)
  if err != nil {
    writeJSON(w, http.StatusUnauthorized, map[string]string{"error":
      "invalid refresh token"})
    return
  }

  // Find user to get current role
  var user *domain.User
  users := s.state.GetAllUsers()
  for _, u := range users {
    if u.ID == userID {
      user = u
      break
    }
  }
  if user == nil {
    writeJSON(w, http.StatusUnauthorized, map[string]string{"error":
      "user not found"})
    return
  }

  tokens, err := s.jwt.GenerateTokenPair(user.ID, user.Role)
  if err != nil {
    writeJSON(w, http.StatusInternalServerError,
      map[string]string{"error": "failed to generate token"})
    return
  }

  writeJSON(w, http.StatusOK, tokens)
}

// --- Resource endpoints ---

// GET /nodes
func (s *Server) getNodes(w http.ResponseWriter, r *http.Request) {
  nodes := s.state.GetAllNodes()
  out := make([]*domain.Node, 0, len(nodes))
  for _, n := range nodes {
    view := *n
    view.Containers = s.state.GetActiveContainersByNode(n.ID)
    out = append(out, &view)
  }
  writeJSON(w, http.StatusOK, out)
```

```go
}

// GET /pools
func (s *Server) getPools(w http.ResponseWriter, r *http.Request) {
  pools := s.state.GetAllPools()
  writeJSON(w, http.StatusOK, pools)
}

// GET /services
func (s *Server) getServices(w http.ResponseWriter, r *http.Request) {
  services := s.state.GetAllServices()
  writeJSON(w, http.StatusOK, services)
}

type CreateServiceRequest struct {
  Name        string            `json:"name"`
  Image       string            `json:"image"`
  RequiredCPU float64           `json:"cpu"`
  RequiredRAM int64             `json:"ram"`
  Replicas    int               `json:"replicas"`
  PoolID      string            `json:"pool"`
  EnvVars     map[string]string `json:"envVars,omitempty"`
  Ports       []int             `json:"ports,omitempty"`
  Cmd         []string          `json:"cmd,omitempty"`
}

// POST /services
func (s *Server) createService(w http.ResponseWriter, r *http.Request) {
  var req CreateServiceRequest
  if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON: " + err.Error()})
    return
  }

  if req.Image == ""  req.Replicas <= 0  req.PoolID == "" {
    writeJSON(w, http.StatusBadRequest, map[string]string{"error": "image, replicas, and pool are required"})
    return
  }

  svc := &domain.Service{
    ID:               generateID("svc"),
    Name:             req.Name,
    Image:            req.Image,
    RequiredCPU:      req.RequiredCPU,
    RequiredRAM:      req.RequiredRAM,
    DesiredReplicas: req.Replicas,
    PoolID:           req.PoolID,
    EnvVars:          req.EnvVars,
    Ports:            req.Ports,
    Cmd:              req.Cmd,
  }
```

```go
    if s.raftNode != nil {
      if err := s.raftNode.Apply(cluster.LogAddService, svc); err != nil {
        writeJSON(w, http.StatusServiceUnavailable, map[string]string{"error": "failed to replicate service: " + err.Error()})
        return
      }
    } else {
      s.state.AddService(svc)
    }

    var containers []map[string]string
    runningCount := 0
    for i := 0; i < req.Replicas; i++ {
      node, err := s.scheduler.Schedule(svc)
      if err != nil {
        log.Printf("api: cannot schedule replica %d of %s: %v", i+1, svc.Name, err)
        c := &domain.Container{
          ID:        generateID("ctr"),
          ServiceID: svc.ID,
          Status:    domain.ContainerPending,
        }
        s.applyAddContainer(c)
        containers = append(containers, map[string]string{"id": c.ID, "status": domain.ContainerPending})
        continue
      }

      // Container scheduled - node found, waiting for Docker to start
      c := &domain.Container{
        ID:        generateID("ctr"),
        ServiceID: svc.ID,
        NodeID:    node.ID,
        Status:    domain.ContainerScheduled,
      }
      s.applyAddContainer(c)

      resp, err := agent.SendCommand(node.Address, agent.Command{
        Type:      agent.CmdStartContainer,
        ServiceID: svc.ID,
        Image:     svc.Image,
        CPULimit:  svc.RequiredCPU,
        RAMLimit:  svc.RequiredRAM,
        EnvVars:   svc.EnvVars,
        Ports:     svc.Ports,
        Cmd:       svc.Cmd,
      })
      if err != nil {
        log.Printf("api: failed to start container on %s: %v", node.ID, err)
        s.applySetContainerStatus(c.ID, domain.ContainerPending)
```

```go
      containers = append(containers, map[string]string{"id": c.ID, "status": domain.ContainerPending})
      continue
    }

    if !resp.Success {
      log.Printf("api: worker %s returned error: %s", node.ID, resp.Error)
      s.applySetContainerStatus(c.ID, domain.ContainerFailed)
      containers = append(containers, map[string]string{"id": c.ID, "status": domain.ContainerFailed, "error": resp.Error})
      continue
    }

    c.DockerID = resp.ContainerID
    c.Status = domain.ContainerRunning
    s.applySetContainerStatus(c.ID, domain.ContainerRunning)
    runningCount++
    containers = append(containers, map[string]string{"id": c.ID, "nodeID": node.ID, "status": domain.ContainerRunning})
    log.Printf("api: service %s replica %d started on node %s", svc.Name, i+1, node.ID)
  }

  s.applySetServiceReplicas(svc.ID, runningCount)

  s.audit.Log("", audit.EventServiceCreate, svc.ID, fmt.Sprintf("service %s created with %d replicas", svc.Name, req.Replicas))

  writeJSON(w, http.StatusCreated, map[string]any{
    "serviceID":  svc.ID,
    "name":       svc.Name,
    "containers": containers,
  })
}

// DELETE /services/{id}
func (s *Server) deleteService(w http.ResponseWriter, r *http.Request) {
  serviceID := r.PathValue("id")

  svc, ok := s.state.GetService(serviceID)
  if !ok {
    writeJSON(w, http.StatusNotFound, map[string]string{"error": "service not found"})
    return
  }

  containers := s.state.GetContainersByService(serviceID)
  for _, c := range containers {
    if c.NodeID != "" {
      node, ok := s.state.GetNode(c.NodeID)
      if ok {
```

```
          agent.SendCommand(node.Address, agent.Command{
            Type:       agent.CmdKillContainers,
            ServiceID: serviceID,
          })
        }
      }
      s.applySetContainerStatus(c.ID, domain.ContainerStopped)
    }

  s.applyRemoveService(serviceID)
  s.audit.Log("", audit.EventServiceDelete, serviceID,
   ↪ fmt.Sprintf("service %s deleted", svc.Name))
  writeJSON(w, http.StatusOK, map[string]string{"status": "deleted"})
}

// GET /audit?action=...&user=...
func (s *Server) getAudit(w http.ResponseWriter, r *http.Request) {
  actionFilter := r.URL.Query().Get("action")
  userFilter := r.URL.Query().Get("user")

  entries := s.state.GetAuditLog(func(e domain.AuditEntry) bool {
    if actionFilter != "" && e.Action != actionFilter {
      return false
    }
    if userFilter != "" && e.UserID != userFilter {
      return false
    }
    return true
  })
  writeJSON(w, http.StatusOK, entries)
}

// applyAddContainer adds a container through Raft or directly.
func (s *Server) applyAddContainer(c *domain.Container) {
  if s.raftNode != nil {
    s.raftNode.Apply(cluster.LogAddContainer, c)
  } else {
    s.state.AddContainer(c)
  }
}

func (s *Server) applySetContainerStatus(id, status string) {
  if s.raftNode != nil {
    s.raftNode.Apply(cluster.LogSetContainerStatus,
     ↪ map[string]string{"id": id, "status": status})
  } else {
    s.state.SetContainerStatus(id, status)
  }
}

// applyRemoveService removes a service through Raft or directly.
func (s *Server) applyRemoveService(id string) {
  if s.raftNode != nil {
```

```go
      s.raftNode.Apply(cluster.LogRemoveService, map[string]string{"id":
       ↪ id})
    } else {
      s.state.RemoveService(id)
    }
}

func (s *Server) applySetServiceReplicas(id string, replicas int) {
    if s.raftNode != nil {
      s.raftNode.Apply(cluster.LogSetServiceReplicas,
       ↪ map[string]any{"id": id, "replicas": replicas})
    } else {
      s.state.SetServiceReplicas(id, replicas)
    }
}

func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    json.NewEncoder(w).Encode(data)
}

func generateID(prefix string) string {
    return fmt.Sprintf("%s-%s", prefix, randomString(8))
}

func randomString(n int) string {
    const letters = "abcdefghijklmnopqrstuvwxyz0123456789"
    b := make([]byte, n)
    for i := range b {
      b[i] = letters[rand.Intn(len(letters))]
    }
    return string(b)
}
```

Listing 9. internal/api/server.go – REST API with JWT middleware

Annex 8. Security – JWT and ACL

```go
package security

import (
  "fmt"
  "time"

  "github.com/golang-jwt/jwt/v5"
)

// TokenPair contains an access token and a refresh token.
type TokenPair struct {
  AccessToken  string `json:"accessToken"`
  RefreshToken string `json:"refreshToken"`
  ExpiresIn    int64  `json:"expiresIn"`
}

// Claims holds the JWT payload.
type Claims struct {
  UserID string `json:"userID"`
  Role   string `json:"role"`
  jwt.RegisteredClaims
}

// RefreshClaims holds the refresh token payload.
type RefreshClaims struct {
  UserID string `json:"userID"`
  jwt.RegisteredClaims
}

// JWTManager handles token generation and validation.
type JWTManager struct {
  secret          []byte
  accessDuration  time.Duration
  refreshDuration time.Duration
}

func NewJWTManager(secret string, accessDuration, refreshDuration time.Duration) *JWTManager {
  return &JWTManager{
    secret:          []byte(secret),
    accessDuration:  accessDuration,
    refreshDuration: refreshDuration,
  }
}

// GenerateTokenPair creates an access token and a refresh token.
func (m *JWTManager) GenerateTokenPair(userID, role string) (*TokenPair, error) {
  // Access token
  now := time.Now()
  accessClaims := Claims{
    UserID: userID,
    Role:   role,
```

```go
		RegisteredClaims: jwt.RegisteredClaims{
			ExpiresAt: jwt.NewNumericDate(now.Add(m.accessDuration)),
			IssuedAt:  jwt.NewNumericDate(now),
		},
	}
	accessToken := jwt.NewWithClaims(jwt.SigningMethodHS256, accessClaims)
	accessStr, err := accessToken.SignedString(m.secret)
	if err != nil {
		return nil, fmt.Errorf("sign access token: %w", err)
	}

	// Refresh token
	refreshClaims := RefreshClaims{
		UserID: userID,
		RegisteredClaims: jwt.RegisteredClaims{
			ExpiresAt: jwt.NewNumericDate(now.Add(m.refreshDuration)),
			IssuedAt:  jwt.NewNumericDate(now),
		},
	}
	refreshToken := jwt.NewWithClaims(jwt.SigningMethodHS256, refreshClaims)
	refreshStr, err := refreshToken.SignedString(m.secret)
	if err != nil {
		return nil, fmt.Errorf("sign refresh token: %w", err)
	}

	return &TokenPair{
		AccessToken:  accessStr,
		RefreshToken: refreshStr,
		ExpiresIn:    int64(m.accessDuration.Seconds()),
	}, nil
}

// ValidateAccessToken parses and validates an access token, returns the claims.
func (m *JWTManager) ValidateAccessToken(tokenStr string) (*Claims, error) {
	token, err := jwt.ParseWithClaims(tokenStr, &Claims{}, func(t *jwt.Token) (interface{}, error) {
		if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
			return nil, fmt.Errorf("unexpected signing method: %v", t.Header["alg"])
		}
		return m.secret, nil
	})
	if err != nil {
		return nil, fmt.Errorf("invalid token: %w", err)
	}

	claims, ok := token.Claims.(*Claims)
	if !ok  !token.Valid {
		return nil, fmt.Errorf("invalid token claims")
	}
```

```go
  return claims, nil
}

// ValidateRefreshToken parses and validates a refresh token, returns the user ID.
func (m *JWTManager) ValidateRefreshToken(tokenStr string) (string, error) {
  token, err := jwt.ParseWithClaims(tokenStr, &RefreshClaims{}, func(t *jwt.Token) (interface{}, error) {
    if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
      return nil, fmt.Errorf("unexpected signing method: %v", t.Header["alg"])
    }
    return m.secret, nil
  })
  if err != nil {
    return "", fmt.Errorf("invalid refresh token: %w", err)
  }

  claims, ok := token.Claims.(*RefreshClaims)
  if !ok  !token.Valid {
    return "", fmt.Errorf("invalid refresh token claims")
  }
  return claims.UserID, nil
}
```

Listing 10. internal/security/jwt.go – JWT token generation and validation

```go
package security

import "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"

// Permission defines an action on a resource.
type Permission struct {
  Resource string // "services", "nodes", "pools", "users", "audit"
  Action   string // "read", "create", "delete"
}

// ACL rules: which roles can perform which actions.
var aclRules = map[string][]Permission{
  domain.RoleAdmin: {
    {Resource: "services", Action: "read"},
    {Resource: "services", Action: "create"},
    {Resource: "services", Action: "delete"},
    {Resource: "nodes", Action: "read"},
    {Resource: "pools", Action: "read"},
    {Resource: "users", Action: "read"},
    {Resource: "users", Action: "create"},
    {Resource: "audit", Action: "read"},
  },
  domain.RoleWriter: {
    {Resource: "services", Action: "read"},
    {Resource: "services", Action: "create"},
```

```go
      {Resource: "services", Action: "delete"},
      {Resource: "nodes", Action: "read"},
      {Resource: "pools", Action: "read"},
      {Resource: "audit", Action: "read"},
    },
    domain.RoleReader: {
      {Resource: "services", Action: "read"},
      {Resource: "nodes", Action: "read"},
      {Resource: "pools", Action: "read"},
      {Resource: "audit", Action: "read"},
    },
}

// CheckPermission returns true if the role has permission for the
  ↪  action on the resource.
func CheckPermission(role, resource, action string) bool {
  perms, ok := aclRules[role]
  if !ok {
    return false
  }
  for _, p := range perms {
    if p.Resource == resource && p.Action == action {
      return true
    }
  }
  return false
}
```

Listing 11. internal/security/acl.go – Role-based access control

Annex 9. Docker Integration

```go
package agent

import (
  "context"
  "fmt"
  "io"
  "log"
  "strconv"
  "sync"

  "github.com/docker/docker/api/types/container"
  "github.com/docker/docker/api/types/image"
  "github.com/docker/docker/client"
  "github.com/docker/go-connections/nat"
)

// DockerManager handles starting and stopping containers via the Docker SDK.
type DockerManager struct {
  cli        *client.Client
  mu         sync.Mutex
  containers map[string][]string // serviceID -> dockerContainerIDs
  seq        int
}

func NewDockerManager() (*DockerManager, error) {
  cli, err := client.NewClientWithOpts(client.FromEnv, client.WithAPIVersionNegotiation())
  if err != nil {
    return nil, fmt.Errorf("docker client: %w", err)
  }
  return &DockerManager{
    cli:        cli,
    containers: make(map[string][]string),
  }, nil
}

// StartContainer pulls the image (if needed) and starts a container.
func (dm *DockerManager) StartContainer(cmd Command) (string, error) {
  ctx := context.Background()

  // Pull image
  log.Printf("docker: pulling image %s", cmd.Image)
  reader, err := dm.cli.ImagePull(ctx, cmd.Image, image.PullOptions{})
  if err != nil {
    return "", fmt.Errorf("pull image %s: %w", cmd.Image, err)
  }
  io.Copy(io.Discard, reader)
  reader.Close()

  // Build environment variables
  var env []string
  for k, v := range cmd.EnvVars {
```

```
    env = append(env, k+"="+v)
  }

  // Build port bindings
  exposedPorts := nat.PortSet{}
  portBindings := nat.PortMap{}
  for _, p := range cmd.Ports {
    port := nat.Port(strconv.Itoa(p) + "/tcp")
    exposedPorts[port] = struct{}{}
    portBindings[port] = []nat.PortBinding{
      {HostIP: "0.0.0.0", HostPort: strconv.Itoa(p)},
    }
  }

  // Container config
  containerConfig := &container.Config{
    Image:        cmd.Image,
    Env:          env,
    Cmd:          cmd.Cmd,
    ExposedPorts: exposedPorts,
  }

  // Host config with resource limits
  hostConfig := &container.HostConfig{
    PortBindings: portBindings,
    Resources: container.Resources{
      NanoCPUs: int64(cmd.CPULimit * 1e9),
      Memory:   cmd.RAMLimit * 1024 * 1024, // MB to bytes
    },
  }

  // Create container
  dm.mu.Lock()
  dm.seq++
  containerName := fmt.Sprintf("dcp-%s-%d", cmd.ServiceID, dm.seq)
  dm.mu.Unlock()
  resp, err := dm.cli.ContainerCreate(ctx, containerConfig, hostConfig,
    nil, nil, containerName)
  if err != nil {
    return "", fmt.Errorf("create container: %w", err)
  }

  // Start container
  if err := dm.cli.ContainerStart(ctx, resp.ID,
    container.StartOptions{}); err != nil {
    return "", fmt.Errorf("start container: %w", err)
  }

  dm.mu.Lock()
  dm.containers[cmd.ServiceID] = append(dm.containers[cmd.ServiceID],
    resp.ID)
  dm.mu.Unlock()
```

```
  log.Printf("docker: started container %s (image=%s, name=%s)", resp.ID[:12], cmd.Image, containerName)
  return resp.ID, nil
}

// KillContainersByService stops and removes containers for a specific service.
// If serviceID is empty, kills all containers.
func (dm *DockerManager) KillContainersByService(serviceID string) error {
  ctx := context.Background()

  dm.mu.Lock()
  var toKill []string
  for svcID, ids := range dm.containers {
    if serviceID == ""  svcID == serviceID {
      toKill = append(toKill, ids...)
      delete(dm.containers, svcID)
    }
  }
  dm.mu.Unlock()

  for _, containerID := range toKill {
    log.Printf("docker: stopping container %s", containerID[:12])
    if err := dm.cli.ContainerStop(ctx, containerID, container.StopOptions{}); err != nil {
      log.Printf("docker: stop %s: %v", containerID[:12], err)
    }
    if err := dm.cli.ContainerRemove(ctx, containerID, container.RemoveOptions{}); err != nil {
      log.Printf("docker: remove %s: %v", containerID[:12], err)
    }
  }

  log.Printf("docker: killed %d container(s) for service=%s", len(toKill), serviceID)
  return nil
}

// KillAllContainers stops and removes all containers managed by this worker.
func (dm *DockerManager) KillAllContainers() error {
  ctx := context.Background()

  dm.mu.Lock()
  var all []string
  for _, ids := range dm.containers {
    all = append(all, ids...)
  }
  dm.containers = make(map[string][]string)
  dm.mu.Unlock()

  for _, containerID := range all {
    log.Printf("docker: stopping container %s", containerID[:12])
```

```
    if err := dm.cli.ContainerStop(ctx, containerID, container.StopOptions{}); err != nil {
      log.Printf("docker: stop %s: %v", containerID[:12], err)
    }
    if err := dm.cli.ContainerRemove(ctx, containerID, container.RemoveOptions{}); err != nil {
      log.Printf("docker: remove %s: %v", containerID[:12], err)
    }
  }

  log.Printf("docker: killed %d container(s)", len(all))
  return nil
}

// RunningContainerIDs returns the IDs of containers managed by this worker.
func (dm *DockerManager) RunningContainerIDs() []string {
  dm.mu.Lock()
  defer dm.mu.Unlock()
  var ids []string
  for _, list := range dm.containers {
    ids = append(ids, list...)
  }
  return ids
}
```

Listing 12. internal/agent/docker.go – Docker container management

Annex 10. Raft FSM

```go
package cluster

import (
  "encoding/json"
  "fmt"
  "io"
  "log"

  "github.com/GeorgeMi/Distributed-Cluster-Platform/internal/domain"
  "github.com/hashicorp/raft"
)

// LogEntry is a command that modifies the cluster state via Raft.
type LogEntry struct {
  Type string          `json:"type"`
  Data json.RawMessage `json:"data"`
}

// Log entry types
const (
  LogAddNode            = "AddNode"
  LogRemoveNode         = "RemoveNode"
  LogSetNodeStatus      = "SetNodeStatus"
  LogAddService         = "AddService"
  LogRemoveService      = "RemoveService"
  LogSetServiceReplicas = "SetServiceReplicas"
  LogAddContainer       = "AddContainer"
  LogRemoveContainer    = "RemoveContainer"
  LogSetContainerStatus = "SetContainerStatus"
)

// FSM implements raft.FSM for the cluster state.
type FSM struct {
  state *State
}

func NewFSM(state *State) *FSM {
  return &FSM{state: state}
}

// Apply is called by Raft when a log entry is committed.
func (f *FSM) Apply(l *raft.Log) any {
  var entry LogEntry
  if err := json.Unmarshal(l.Data, &entry); err != nil {
    log.Printf("fsm: unmarshal error: %v", err)
    return err
  }

  switch entry.Type {
  case LogAddNode:
    var node domain.Node
    json.Unmarshal(entry.Data, &node)
    f.state.AddNode(&node)
```

```go
      f.state.AssignNodeToPool(node.ID, &node)
      log.Printf("fsm: apply AddNode %s", node.ID)

    case LogRemoveNode:
      var args struct {
        ID string `json:"id"`
      }
      json.Unmarshal(entry.Data, &args)
      f.state.RemoveNode(args.ID)

    case LogSetNodeStatus:
      var args struct {
        ID     string `json:"id"`
        Status string `json:"status"`
      }
      json.Unmarshal(entry.Data, &args)
      f.state.SetNodeStatus(args.ID, args.Status)

    case LogAddService:
      var svc domain.Service
      json.Unmarshal(entry.Data, &svc)
      f.state.AddService(&svc)
      log.Printf("fsm: apply AddService %s", svc.Name)

    case LogRemoveService:
      var args struct {
        ID string `json:"id"`
      }
      json.Unmarshal(entry.Data, &args)
      f.state.RemoveService(args.ID)

    case LogSetServiceReplicas:
      var args struct {
        ID       string `json:"id"`
        Replicas int    `json:"replicas"`
      }
      json.Unmarshal(entry.Data, &args)
      f.state.SetServiceReplicas(args.ID, args.Replicas)

    case LogAddContainer:
      var c domain.Container
      json.Unmarshal(entry.Data, &c)
      f.state.AddContainer(&c)

    case LogRemoveContainer:
      var args struct {
        ID string `json:"id"`
      }
      json.Unmarshal(entry.Data, &args)
      f.state.RemoveContainer(args.ID)

    case LogSetContainerStatus:
      var args struct {
        ID     string `json:"id"`
```

```go
			Status string `json:"status"`
		}
		json.Unmarshal(entry.Data, &args)
		f.state.SetContainerStatus(args.ID, args.Status)

	default:
		log.Printf("fsm: unknown entry type: %s", entry.Type)
	}

	return nil
}

func (f *FSM) Snapshot() (raft.FSMSnapshot, error) {
	return &fsmSnapshot{data: f.state.Snapshot()}, nil
}

func (f *FSM) Restore(rc io.ReadCloser) error {
	defer rc.Close()
	var snap Snapshot
	if err := json.NewDecoder(rc).Decode(&snap); err != nil {
		return fmt.Errorf("snapshot decode: %w", err)
	}
	f.state.Restore(snap)
	log.Printf("fsm: restored state (%d nodes, %d services, %d containers)",
		len(snap.Nodes), len(snap.Services), len(snap.Containers))
	return nil
}

type fsmSnapshot struct {
	data Snapshot
}

func (s *fsmSnapshot) Persist(sink raft.SnapshotSink) error {
	defer sink.Close()
	data, err := json.Marshal(s.data)
	if err != nil {
		sink.Cancel()
		return fmt.Errorf("snapshot marshal: %w", err)
	}
	if _, err := sink.Write(data); err != nil {
		sink.Cancel()
		return fmt.Errorf("snapshot write: %w", err)
	}
	return nil
}

func (s *fsmSnapshot) Release() {}
```

Listing 13. internal/cluster/fsm.go – Raft Finite State Machine